Database Systems Terminology & Concepts Reference

A personal reference for understanding database concepts, not just memorizing them.

Contents

A.1 Introduction to Database Systems **2**
- A.1.1 What is a DBMS? 2
- A.1.2 Why DBMS over File Systems? 2
- A.1.3 Data Models 2
- A.1.4 Three Levels of Abstraction 2
- A.1.5 Schema vs Instance 3
- A.1.6 Database Languages: DDL and DML 3
- A.1.7 Database Engine Components 3

A.2 Relational Algebra **3**
- A.2.1 Relational Model Terminology 3
- A.2.2 Six Basic Operations 4
- A.2.3 Join Operations 6
- A.2.4 Set Operations 7
- A.2.5 Rename and Assignment 8
- A.2.6 Aggregation 8
- A.2.7 Operation Properties 9
- A.2.8 Query Building Patterns 9

A.3 SQL Fundamentals **10**
- A.3.1 SQL Components 10
- A.3.2 SQL Syntax Rules 10
- A.3.3 SQL Data Types 10

A.4 Data Definition Language (DDL) **10**
- A.4.1 CREATE TABLE 10
- A.4.2 Integrity Constraints 11
- A.4.3 Modifying Tables 11

A.5 Basic Query Structure **11**
- A.5.1 SELECT-FROM-WHERE 11
- A.5.2 The SELECT Clause 12
- A.5.3 DISTINCT vs ALL 12
- A.5.4 The WHERE Clause 13
- A.5.5 NULL Values 14

A.6 Multi-Table Queries **14**
- A.6.1 Cartesian Product and Joins 14
- A.6.2 Table Aliases (AS) 15

A.7 String Operations **16**
- A.7.1 LIKE Pattern Matching 16
- A.7.2 String Functions 16

A.8 ORDER BY Clause **16**

A.9 Set Operations **16**

A.10 Aggregate Functions **17**
A.11 GROUP BY and HAVING **18**
A.11.1 GROUP BY Clause 18
A.11.2 HAVING Clause 19
A.11.3 WHERE vs HAVING: Key Difference 19
A.12 Query Execution Order **20**
A.13 Nested Subqueries **20**
A.13.1 IN and NOT IN 20
A.13.2 SOME/ANY and ALL 21
A.13.3 EXISTS and NOT EXISTS 21
A.14 Common SQL Mistakes **22**
A.15 SQL Quick Reference **22**
A.16 SQL Advanced (L04) **22**
A.16.1 NULL Values 22
A.16.1.1 Three-Valued Logic 22
A.16.1.2 Testing for NULL 23
A.16.1.3 NULL in Aggregates 23
A.16.2 Data Modification 23
A.16.2.1 INSERT 24
A.16.2.2 DELETE 24
A.16.2.3 UPDATE 24
A.16.3 Join Expressions 25
A.16.3.1 Join Conditions 25
A.16.3.2 Outer Join Examples 26
A.16.4 Set Membership (IN, NOT IN) 27
A.16.5 Set Comparison (SOME, ALL) 27
A.16.6 Advanced Subqueries 28
A.16.6.1 Scalar Subqueries 28
A.16.6.2 Correlated Subqueries 28
A.16.6.3 EXISTS and NOT EXISTS 29
A.16.6.4 Derived Relations (Subqueries in FROM) 29
A.16.6.5 WITH Clause (Common Table Expressions) 30
A.16.7 Views 30
A.16.7.1 Creating Views 30
A.16.7.2 View Expansion 30
A.16.7.3 Updatable Views 31
A.16.7.4 Materialized Views 31
A.16.8 Integrity Constraints 31
A.16.8.1 All Constraint Types Overview 31
A.16.8.2 Single-Relation Constraints 32
A.16.8.3 Referential Integrity (Foreign Keys) 32
A.16.8.4 Referential Actions 32
A.16.8.5 Complex CHECK Constraints 33
A.16.8.6 Assertions 34
A.16.9 Transactions 34
A.16.9.1 Transaction Control Statements 34
A.16.10 Common Patterns from Exercises 34
A.16.10.1 Pattern: Finding Maximum/Minimum 35
A.16.10.2 Pattern: Counting with Conditions 35
A.16.10.3 Pattern: NOT EXISTS for “All” Queries 35
A.16.10.4 Pattern: Self-Join for Comparisons 35
A.16.11 Summary: SQL Advanced Quick Reference 35

A.17 Entity-Relationship Model (L05) . . . 36
A.17.1 ER Model Basics . . . 36
A.17.1.1 Core Concepts . . . 36
A.17.1.2 Finding Entities and Relationships . . . 36
A.17.1.3 Attribute Types . . . 36
A.17.1.4 Primary Keys . . . 36
A.17.2 Relationships . . . 36
A.17.2.1 Relationship Basics . . . 36
A.17.2.2 Roles (Recursive Relationships) . . . 37
A.17.2.3 Relationship Attributes . . . 37
A.17.3 Mapping Cardinality . . . 37
A.17.3.1 Four Types of Binary Relationships . . . 37
A.17.3.2 Three Notations (Know All Three!) . . . 37
A.17.4 Participation Constraints . . . 38
A.17.5 Keys for Relationship Sets . . . 38
A.17.6 Weak Entity Types . . . 38
A.17.6.1 Definition . . . 38
A.17.6.2 Discriminator (Partial Key) . . . 39
A.17.6.3 ER Notation for Weak Entities . . . 39
A.17.6.4 Rules for Weak Entities . . . 39
A.17.7 Specialization (ISA Hierarchy) . . . 39
A.17.7.1 Concept . . . 39
A.17.7.2 Attribute Inheritance . . . 39
A.17.7.3 Overlapping vs Disjoint . . . 40
A.17.7.4 Total vs Partial Specialization . . . 40
A.17.8 ER Design Process . . . 40
A.17.8.1 Steps to Design an ER Diagram . . . 40
A.17.8.2 Design Goals . . . 40
A.17.9 Exercise Pattern: Car Insurance Example . . . 40
A.17.10 Summary: ER Notation Quick Reference . . . 40
A.18 ER to Relational Mapping (L06) . . . 41
A.18.1 The 7-Step Mapping Algorithm . . . 41
A.18.2 M1: Mapping Regular Entity Types . . . 41
A.18.3 M2: Mapping Weak Entity Types . . . 41
A.18.4 M3: Mapping Binary M:N Relationships . . . 42
A.18.5 M4: Mapping Binary 1:N Relationships . . . 42
A.18.6 M5: Mapping Binary 1:1 Relationships . . . 43
A.18.7 M6: Complex Relationships . . . 43
A.18.7.1 Recursive (Self-Referencing) Relationships . . . 43
A.18.7.2 Ternary Relationships . . . 44
A.18.8 M7: Complex Attributes . . . 44
A.18.8.1 Multivalued Attributes . . . 44
A.18.8.2 Composite Attributes . . . 44
A.18.8.3 Derived Attributes . . . 44
A.18.9 Mapping Specialization (ISA) . . . 44
A.18.9.1 Alternative 1: Main Classes . . . 45
A.18.9.2 Alternative 2: Partitioning . . . 45
A.18.9.3 Alternative 3: Full Redundancy . . . 45
A.18.9.4 Alternative 4: Single Relation . . . 45
A.18.9.5 Choosing an Alternative . . . 45
A.18.10 Exercise Pattern: Car Insurance Mapping . . . 46
A.18.11 Summary: Quick Mapping Reference . . . 46
A.19 Normalization (L07) . . . 46

A.19.1 Why Normalize? . . . 46
A.19.2 Functional Dependencies . . . 47
A.19.2.1 Trivial vs Non-trivial . . . 47
A.19.2.2 Armstrong's Axioms . . . 47
A.19.2.3 Derived Rules . . . 47
A.19.3 Deriving FDs with Armstrong's Axioms . . . 47
A.19.4 Attribute Closure . . . 49
A.19.4.1 Closure Algorithm . . . 49
A.19.4.2 Example . . . 49
A.19.4.3 Using Closure to Verify FDs . . . 50
A.19.5 Keys . . . 50
A.19.5.1 Prime vs Non-Prime Attributes . . . 50
A.19.5.2 Finding Candidate Keys . . . 50
A.19.6 Normal Forms . . . 51
A.19.6.1 First Normal Form (1NF) . . . 51
A.19.6.2 Second Normal Form (2NF) . . . 51
A.19.6.3 Third Normal Form (3NF) . . . 52
A.19.6.4 BCNF (Boyce-Codd Normal Form) . . . 54
A.19.6.5 3NF vs BCNF: The Key Difference . . . 55
A.19.6.6 Checking Normal Forms: Quick Decision Flowchart . . . 55
A.19.6.7 Red Flag Patterns . . . 56
A.19.7 Decomposition . . . 56
A.19.7.1 Lossless Decomposition . . . 56
A.19.7.2 Dependency Preserving . . . 58
A.19.7.3 3NF vs BCNF Trade-off . . . 58
A.19.8 Summary: Quick Reference . . . 58
A.20 Physical Design & Indexing (L08) . . . 59
A.20.1 Storage Concepts . . . 59
A.20.1.1 Storage Hierarchy . . . 59
A.20.1.2 File Organization Types . . . 60
A.20.2 Index Fundamentals . . . 60
A.20.2.1 Primary vs Secondary Index . . . 60
A.20.2.2 Dense vs Sparse Index . . . 60
A.20.3 B+ Tree Structure . . . 60
A.20.3.1 Node Structure . . . 60
A.20.3.2 B+ Tree Properties (Degree n) . . . 60
A.20.3.3 Common Degree Values . . . 61
A.20.3.4 Navigation . . . 61
A.20.3.5 Search Operations . . . 61
A.20.4 B+ Tree Insert . . . 61
A.20.4.1 Split Leaf . . . 61
A.20.4.2 Split Internal Node . . . 62
A.20.5 B+ Tree Delete . . . 63
A.20.5.1 Redistribute . . . 63
A.20.5.2 Merge . . . 63
A.20.5.3 Underflow Thresholds . . . 63
A.20.6 Hashing . . . 64
A.20.6.1 Concepts . . . 64
A.20.6.2 Hashing vs B+ Tree . . . 64
A.20.7 Summary: Quick Reference . . . 64
A.21 Query Processing (L09) . . . 64
A.21.1 Query Processing Overview . . . 64
A.21.1.1 Processing Steps . . . 65
A.21.2 External Merge Sort . . . 65

A.21.2.1 Algorithm . . . 65
A.21.2.2 Cost Formula . . . 65
A.21.2.3 Example Calculation . . . 65
A.21.3 Join Algorithms . . . 65
A.21.3.1 Notation . . . 66
A.21.3.2 Nested-Loop Join (Simple) . . . 66
A.21.3.3 Block Nested-Loop Join . . . 66
A.21.3.4 Sort-Merge Join . . . 67
A.21.3.5 Hash Join . . . 67
A.21.3.6 Join Algorithm Comparison . . . 67
A.21.4 Selection Algorithms . . . 67
A.21.4.1 Selection Methods . . . 67
A.21.5 Cost Estimation . . . 67
A.21.5.1 Catalog Statistics . . . 68
A.21.6 Size Estimation . . . 68
A.21.6.1 Selection Size . . . 68
A.21.6.2 Join Size . . . 68
A.21.6.3 Projection Size . . . 68
A.21.7 Query Optimization . . . 68
A.21.7.1 Equivalence Rules . . . 69
A.21.7.2 Heuristic Optimization . . . 69
A.21.8 Pipelining . . . 69
A.21.9 Summary: Quick Reference . . . 69
A.22 Transactions & Concurrency (L10) . . . 69
A.22.1 Transaction Fundamentals . . . 70
A.22.1.1 Example Transaction . . . 70
A.22.1.2 Transaction Operations . . . 70
A.22.2 ACID Properties . . . 70
A.22.2.1 Which Property is Violated? . . . 70
A.22.3 Concurrency Problems . . . 70
A.22.4 Schedules and Serializability . . . 71
A.22.4.1 Definitions . . . 71
A.22.5 Conflict Serializability . . . 71
A.22.5.1 Conflict Table . . . 71
A.22.5.2 Definition . . . 71
A.22.6 Precedence Graph . . . 71
A.22.6.1 Construction Algorithm . . . 71
A.22.6.2 Result Interpretation . . . 71
A.22.6.3 Example . . . 72
A.22.7 Lock-Based Concurrency Control . . . 72
A.22.7.1 Lock Types . . . 72
A.22.7.2 Lock Compatibility Matrix . . . 72
A.22.7.3 Lock Operations . . . 72
A.22.8 Two-Phase Locking (2PL) . . . 72
A.22.8.1 The Two Phases . . . 73
A.22.8.2 2PL Examples . . . 73
A.22.8.3 2PL Variants . . . 73
A.22.9 Deadlock . . . 73
A.22.9.1 Example . . . 73
A.22.9.2 Wait-For Graph . . . 73
A.22.9.3 Example Wait-For Graph . . . 74
A.22.9.4 Deadlock Handling . . . 74
A.22.9.5 Victim Selection . . . 74
A.22.10 Deadlock Exercise Pattern . . . 74

A.22.11 Summary: Quick Reference . . . 74
A.23 Recovery (L11) . . . 75
A.23.1 Recovery Purpose . . . 75
A.23.2 Failure Types . . . 75
A.23.3 Write-Ahead Logging (WAL) . . . 75
A.23.4 Log Record Structure . . . 75
A.23.5 Recovery Algorithm . . . 76
A.23.6 Complete Recovery Example . . . 76
A.23.7 Checkpointing . . . 77
A.23.8 Summary: Quick Reference . . . 77
B.1 Relational Algebra Reference . . . 79
B.1.1 Operators at a Glance . . . 79
B.1.2 Set vs Bag Semantics . . . 79
B.1.3 Aggregation . . . 79
B.1.4 Division . . . 79
B.1.5 Key Properties . . . 79
B.2 SQL Reference . . . 79
B.2.1 Basic Query Structure . . . 79
B.2.2 Set Operations . . . 80
B.2.3 NULL Handling . . . 80
B.2.4 Join Types . . . 80
B.2.5 Aggregate Functions . . . 80
B.2.6 Subquery Operators . . . 80
B.2.7 Constraints . . . 80
B.3 Normalization Reference . . . 80
B.3.1 Armstrong's Axioms . . . 81
B.3.2 Closure Algorithm . . . 81
B.3.3 Key Definitions . . . 81
B.3.4 Normal Forms . . . 81
B.3.5 Decomposition Tests . . . 81
B.3.6 Lossless Decomposition Test . . . 81
B.3.7 BCNF vs 3NF . . . 82
B.4 B+ Tree Reference . . . 82
B.4.1 Node Capacity (Degree n) . . . 82
B.4.2 Common Degree Values . . . 82
B.4.3 Insert Algorithm . . . 82
B.4.4 Delete Algorithm . . . 82
B.4.5 Underflow Conditions . . . 82
B.4.6 Index Types . . . 83
B.5 Query Cost Reference . . . 83
B.5.1 Notation . . . 83
B.5.2 External Sort . . . 83
B.5.3 Join Algorithms . . . 83
B.5.4 Selection Size Estimation . . . 83
B.5.5 Join Size Estimation . . . 84
B.5.6 Optimization Heuristics . . . 84
B.6 ACID & Locking Reference . . . 84
B.6.1 ACID Properties . . . 84
B.6.2 Conflict Definition . . . 84
B.6.3 Precedence Graph . . . 84

B.6.4 Lock Compatibility 84
B.6.5 Two-Phase Locking (2PL) 84
B.6.6 Wait-For Graph 84
B.6.7 Recovery Log Format 84
B.6.8 Recovery Algorithm 84
B.6.9 Exam Traps 84

PART A: Course Overview & Key Definitions

A.1 Introduction to Database Systems

A.1.1 What is a DBMS?

Database Management System (DBMS)

A DBMS is a **collection of interrelated data** combined with **programs to access that data**. It provides a convenient and efficient environment for storing and retrieving data.
(INTUITION) The intuition is that a DBMS is much more than storage—it **enforces rules**, **handles concurrent access**, and **guarantees data integrity**. When data is valuable, large, and shared by multiple users, you need a system that automatically enforces constraints that a simple file system cannot.

A.1.2 Why DBMS over File Systems?

File-based systems suffer from several problems that DBMS solves:

Problems with File Systems (CRITICAL)

1. **Data Redundancy & Inconsistency**: Data stored in multiple formats leads to duplication and inconsistencies.
2. **Data Isolation**: Multiple files and formats make integration difficult.
3. **Difficulty Accessing Data**: Need new programs for each new query/task.
4. **Integrity Problems**: Constraints (e.g., balance > 0) are buried in application code.
5. **Atomicity of Updates**: Failures may leave database in partial (inconsistent) state.
6. **Concurrent Access**: Uncontrolled access by multiple users causes inconsistencies.
7. **Security Problems**: Hard to provide selective access to data.

Origin of ACID Properties (KEY CONCEPT)

The four ACID properties directly solve these file system problems:

- **Atomicity** $\rightarrow$ solves partial update problem (all-or-nothing)
- **Consistency** $\rightarrow$ solves integrity constraint problem
- **Isolation** $\rightarrow$ solves concurrent access problem
- **Durability** $\rightarrow$ solves failure/recovery problem

Understanding this connection helps you see *why* ACID exists, not just what it means.

A.1.3 Data Models

Data Models

A data model defines how data is structured and relationships are represented:

- **Relational Model**: Data stored in tables (rows and columns)—*primary focus of this course*
- **Entity-Relationship Model**: Used for database design
- **Semi-structured Model**: XML, JSON
- **Graph Model**: Nodes and edges
- Historical: Network model, Hierarchical model

A.1.4 Three Levels of Abstraction

Database Abstraction Levels (KEY CONCEPT)

1. **Physical Level**: How data is *actually stored* (files, indexes, storage structures). Hidden from most users.
2. **Logical Level**: What data exists and relationships between entities. This is the *schema definition*.
3. **View Level**: What specific users/applications see. Can hide data for security or simplicity.

Data Independence (REMEMBER)

The intuition behind these levels is **data independence**:

- You can change *how* data is physically stored without changing the logical schema
- You can change the logical schema without breaking all applications (via views)

This separation is powerful—it allows systems to evolve without cascading changes.

A.1.5 Schema vs Instance

Schema vs Instance Analogy

- **Schema** = structure/type definition (changes rarely)
- **Instance** = actual data at a point in time (changes frequently)

Analogy: Schema is like a variable's *type*, instance is like the variable's *value*.

A.1.6 Database Languages: DDL and DML

DDL - Data Definition Language

Defines the database **schema** (structure):

- `CREATE TABLE` - creates new tables
- `ALTER TABLE` - modifies table structure
- `DROP TABLE` - removes tables

DDL generates metadata stored in the *data dictionary*: schema definitions, integrity constraints, authorization info.

DML - Data Manipulation Language

Accesses and **manipulates data**:

- `SELECT` - retrieves data (query)
- `INSERT` - adds new data
- `UPDATE` - modifies existing data
- `DELETE` - removes data

Declarative vs Procedural (KEY CONCEPT)

SQL is **declarative**—you specify *what* you want, not *how* to get it.

Why this matters: Because SQL is declarative, the DBMS can *optimize* your query. You focus on the logic, the query optimizer figures out the best execution plan. This is why query optimization (L09) exists!

A.1.7 Database Engine Components

DBMS Architecture Components

1. **Storage Manager**: Handles file storage, buffer management, file organization
2. **Query Processor**: Parses, optimizes, and executes queries
3. **Transaction Manager**: Ensures ACID properties, handles concurrency and recovery

These components are covered in detail in later lectures (L08, L09, L10, L11).

A.2 Relational Algebra

A.2.1 Relational Model Terminology

Core Terms (CRITICAL)

- **Relation** = Table (a set of tuples)
- **Tuple** = Row (a single record)
- **Attribute** = Column (a property)
- **Domain** = Set of permitted values for an attribute
- **Schema** = Structure definition, e.g., `Instructor(ID, name, dept, salary)`
- **Instance** = Actual data at a specific point in time

Keys (CRITICAL)

Key Hierarchy: Superkey $\supseteq$ Candidate Key $\supseteq$ Primary Key

- **Superkey:** Any set of attributes that uniquely identifies tuples. May contain "extra" attributes.
- **Candidate Key:** Minimal superkey—remove any attribute and it loses uniqueness.
- **Primary Key:** The chosen candidate key (underlined in schema).
- **Foreign Key:** Attribute in one relation referencing the primary key of another.

Understanding Keys (INTUITION)

The intuition behind keys: A superkey can have "extra baggage"—any superset of a candidate key works. A candidate key is the most compact form that still uniquely identifies rows. The primary key is just which candidate key we choose as "the official one."

Example: If {ID} is a candidate key, then {ID}, {ID, name}, {ID, name, dept} are all superkeys, but only {ID} is a candidate key (minimal).

A.2.2 Six Basic Operations

SELECT (σ) — Filter Rows (CRITICAL)

Notation: $\sigma_{condition}(R)$
Purpose: Filter **ROWS** where condition is true (horizontal slice).
Operators: $=$, $\neq$, $<$, $>$, $\leq$, $\geq$, $\wedge$ (AND), $\vee$ (OR), $\neg$ (NOT)
Use when you see: "older than", "greater than", "lives in", "who are", "that are"

SELECT Examples with Input/Output

Input: Student

ID	name	age
1	Ali	20
2	Bob	22
3	Cat	20

Query: $\sigma_{age=20}(\text{Student})$
Plain English: Select all rows from Student where age equals 20.
Output:

ID	name	age
1	Ali	20
3	Cat	20

Query: $\sigma_{age>20 \wedge ID<3}(\text{Student})$
Plain English: Select rows where age is greater than 20 AND ID is less than 3.
Output:

ID	name	age
2	Bob	22

PROJECT (Π) — Select Columns (CRITICAL)

Notation: $\Pi_{A_1,A_2,\ldots}(R)$
Purpose: Keep only specified **COLUMNS** (vertical slice). **Duplicates are removed!**

PROJECT Examples with Input/Output

Input: Student

ID	name	age
1	Ali	20
2	Bob	22
3	Cat	20

Query: $\Pi_{name}(\text{Student})$
Plain English: Keep only the name column from Student, removing duplicates.
Output:

name
Ali
Bob
Cat

Query: $\Pi_{age}(\text{Student})$
Plain English: Keep only the age column, removing duplicate values.
Output:

age
20
22

Only 2 rows! Age 20 appeared twice but duplicates are removed.

Composition Order (CRITICAL)

SELECT before PROJECT if filtering attribute is needed!
(REMEMBER) Pattern: $\Pi_{output_cols}(\sigma_{condition}(relation))$

Composition Order — Correct vs Wrong

Input: Student

ID	name	age
1	Ali	20
2	Bob	22
3	Cat	20

Goal: Get names of students aged 20.
CORRECT: $\Pi_{name}(\sigma_{age=20}(\text{Student}))$
Plain English: First filter to students aged 20, then keep only their names.

name
Ali
Cat

WRONG: $\sigma_{age=20}(\Pi_{name}(\text{Student}))$ — ERROR! After projecting only name, the `age` column is gone and we cannot filter by it.

CARTESIAN PRODUCT ($\times$)

Notation: $R \times S$
Purpose: Combine EVERY row from R with EVERY row from S.
Result size: $|R| \times |S|$ rows. Creates very large tables!
(REMEMBER) Rarely used alone—usually combined with SELECT to filter meaningful combinations (this becomes a join).

CARTESIAN PRODUCT Example

Input: Student

ID	name
1	Ali
2	Bob

Input: Course

cID	title
C1	Math
C2	DB

Query: Student $\times$ Course
Plain English: Combine every student row with every course row (Cartesian product).
Output: ($2 \times 2 = 4$ rows)

ID	name	cID	title
1	Ali	C1	Math
1	Ali	C2	DB
2	Bob	C1	Math
2	Bob	C2	DB

A.2.3 Join Operations

Theta Join ($\bowtie_\theta$) (CRITICAL)

Notation: $R \bowtie_\theta S$
Definition: $R \bowtie_\theta S = \sigma_\theta(R \times S)$
Purpose: Combine tables based on condition θ.
When θ uses only $=$, this is called an **equijoin**.

Theta Join Example

Input: Student

sID	name
1	Ali
2	Bob

Input: Enrolled

sID	course
1	Math
1	DB
3	DB

Query: Student $\bowtie_{\text{Student.sID=Enrolled.sID}}$ Enrolled
Plain English: Join Student and Enrolled where their sID values match (keeps both sID columns).
Output: (only matching sID values)

Student.sID	name	Enrolled.sID	course
1	Ali	1	Math
1	Ali	1	DB

Note: Bob (sID=2) and sID=3 enrollment are excluded—no match!
Note: Theta join keeps **both** sID columns.

Natural Join ($\bowtie$) (CRITICAL)

Notation: $R \bowtie S$ (no subscript)
Purpose: Automatically join on **ALL common attribute names**.
Key difference from Theta Join: Common attributes appear **ONLY ONCE** in result.

Natural Join Example

Using same **Student** and **Enrolled** tables from above.
Query: Student $\bowtie$ Enrolled
Plain English: Join Student and Enrolled on their common attribute (sID), keeping one copy of the join column.
Output:

sID	name	course
1	Ali	Math
1	Ali	DB

Note: Only **ONE** sID column (merged), unlike theta join which has two.

Natural Join Danger (REMEMBER)

Natural join uses **ALL** common attributes—this may over-constrain your query if tables share attribute names unintentionally.
Always verify which attributes are being matched!

Outer Joins — Preserve Non-Matching Tuples

Left Outer Join: Keep ALL from R, NULL for non-matching S
Right Outer Join: Keep ALL from S, NULL for non-matching R
Full Outer Join: Keep ALL from both, NULL where no match

Outer Join Examples

Input: Student

sID	name
1	Ali
2	Bob

Input: Enrolled

sID	course
1	Math
3	DB

LEFT OUTER JOIN: Student $\bowtie$ Enrolled
Plain English: Keep ALL students; if no matching enrollment, fill with NULL.

sID	name	course
1	Ali	Math
2	Bob	NULL

Bob has no enrollment $\rightarrow$ NULL for course.
RIGHT OUTER JOIN: Student $\bowtie$ Enrolled
Plain English: Keep ALL enrollments; if no matching student, fill with NULL.

sID	name	course
1	Ali	Math
3	NULL	DB

sID=3 has no student $\rightarrow$ NULL for name.
FULL OUTER JOIN: Student $\bowtie$ Enrolled
Plain English: Keep ALL rows from both tables; fill missing values with NULL.

sID	name	course
1	Ali	Math
2	Bob	NULL
3	NULL	DB

A.2.4 Set Operations

Compatibility Requirement (CRITICAL)

For $\cup$, $\cap$, $-$ to work, relations must be **union-compatible**:
1. Same arity (same number of attributes)
2. Compatible domains (same types in same positions)

Union, Intersection, Difference (CRITICAL)

UNION $R \cup S$: All tuples in R OR S (duplicates removed). **Commutative.**
INTERSECTION $R \cap S$: Tuples in BOTH R AND S. **Commutative.**
DIFFERENCE $R - S$: Tuples in R but NOT in S. **NOT commutative!**
Use when you see: "does not", "have not", "except", "excluding", "never"
(REMEMBER) $R - S \neq S - R$ — Order matters for set difference!

Set Operations Examples

Input: MathStudents (students in Math)

name
Ali
Bob

Input: DBStudents (students in DB)

name
Bob
Cat

UNION: MathStudents $\cup$ DBStudents
Plain English: All students who are in Math OR DB (or both), duplicates removed.

name
Ali
Bob
Cat

Bob appears only once (duplicates removed).
INTERSECTION: MathStudents $\cap$ DBStudents
Plain English: Only students who are in BOTH Math AND DB.

name
Bob

DIFFERENCE: MathStudents $-$ DBStudents
Plain English: Students in Math but NOT in DB.

name
Ali

DIFFERENCE (reversed): DBStudents $-$ MathStudents
Plain English: Students in DB but NOT in Math.

name
Cat

Note: $R - S \neq S - R$!

Pattern for "NOT" Queries (CRITICAL)

Problem: Find employees NOT working for BigBank.
Solution using Set Difference:

$$\Pi_{eID,name}(employee) - \Pi_{eID,name}(employee \bowtie \sigma_{company=\text{"BigBank"}}(works))$$

(KEY CONCEPT) All employees MINUS those at BigBank = employees NOT at BigBank. This is the standard pattern for "NOT" or "except" queries in RA.

A.2.5 Rename and Assignment

Rename (ρ) and Assignment ($\leftarrow$)

Rename: $\rho_x(E)$ or $\rho_{x(A_1,\ldots,A_n)}(E)$ — Give result or attributes new names.
Assignment: $temp \leftarrow E$ — Store intermediate results.
Example:

$$\begin{aligned} Physics &\leftarrow \sigma_{dept=\text{"Physics"}}(instructor) \\ Music &\leftarrow \sigma_{dept=\text{"Music"}}(instructor) \\ Result &\leftarrow Physics \cup Music \end{aligned}$$

A.2.6 Aggregation

Aggregation (γ)

Notation: $\gamma_{G;F}(R)$ where G = grouping attributes, F = aggregate functions.
Functions: count(*), count(attr), sum(attr), avg(attr), min(attr), max(attr)

Aggregation Examples

Input: Student

ID	dept	age
1	CS	20
2	CS	22
3	Math	21
4	Math	23

Query: $\gamma_{;count(*)}(\text{Student})$
Plain English: Count the total number of rows in Student (no grouping).
Output:

count(*)
4

Query: $\gamma_{dept;count(*),avg(age)}(\text{Student})$
Plain English: Group by department, then compute count and average age for each group.
Output:

dept	count(*)	avg(age)
CS	2	21
Math	2	22

Query: $\gamma_{dept;min(age),max(age)}(\text{Student})$
Plain English: Group by department, then find minimum and maximum age in each group.
Output:

dept	min(age)	max(age)
CS	20	22
Math	21	23

Aggregation Intuition (INTUITION)

Think of γ as two steps:
1. **Group:** Partition tuples by the grouping attributes
2. **Compute:** For each group, compute the aggregate function(s)

Each group produces **ONE output row**.

A.2.7 Operation Properties

Commutativity (REMEMBER)

Commutative (order doesn't matter):
- SELECT compositions: $\sigma_p(\sigma_q(R)) = \sigma_q(\sigma_p(R)) = \sigma_{p \wedge q}(R)$
- UNION: $R \cup S = S \cup R$
- INTERSECTION: $R \cap S = S \cap R$
- Cartesian Product: $R \times S = S \times R$
- Natural Join: $R \bowtie S = S \bowtie R$

NOT Commutative:
- SET DIFFERENCE: $R - S \neq S - R$

A.2.8 Query Building Patterns

Common Query Patterns (CRITICAL)

Pattern 1 — Simple Query:
"Find X where Y" $\Rightarrow \Pi_X(\sigma_Y(R))$
Pattern 2 — Join Query:
"Find X from R and S where condition" $\Rightarrow \Pi_X(\sigma_{cond}(R \bowtie S))$
Pattern 3 — Negation ("NOT"):
"Find X that are NOT Y" $\Rightarrow \Pi_{attrs}(R) - \Pi_{attrs}(\sigma_Y(R))$

Pattern 4 — Universal ("ALL"):
"Find X that satisfy ALL Y" ⇒ Use division or double negation.

Common Mistakes to Avoid

1. **Project before Select:** Don't remove attributes you need for filtering!
2. **Forget union-compatibility:** Set operations need same schema.
3. **Assume difference is commutative:** $R - S \neq S - R$!
4. **Natural join danger:** Unintended matches on common attribute names.
5. **Forget to project:** Join results often need cleanup.

A.3 SQL Fundamentals

SQL Overview (CRITICAL)

SQL (Structured Query Language) is the standard language for relational databases. Understanding SQL is essential for nearly every exam question.

A.3.1 SQL Components

SQL Sub-Languages

SQL consists of several sub-languages:

- **DDL (Data Definition Language)**: Define schemas, create/modify/delete tables
- **DML (Data Manipulation Language)**: Query and modify data
- **Integrity**: Specify constraints (PRIMARY KEY, FOREIGN KEY)
- **View Definition**: Create virtual tables
- **Transaction Control**: COMMIT, ROLLBACK

A.3.2 SQL Syntax Rules

Case Sensitivity (CRITICAL)

- **Keywords are case-insensitive**: `SELECT` = `select` = `SeLeCt`
- **String constants ARE case-sensitive**: `'FRED'` ≠ `'fred'`
- **Use single quotes for strings**: `'abc'` (NOT `"abc"`)

Syntax (REMEMBER)

All SQL statements end with a semicolon (`;`).

A.3.3 SQL Data Types

SQL Data Types

Type	Description	Example
`char(n)`	Fixed-length string (padded)	`char(5)` stores exactly 5 chars
`varchar(n)`	Variable-length string	`varchar(20)` up to 20 chars
`int`	Integer	Machine-dependent
`numeric(p,d)`	Fixed-point decimal	`numeric(8,2)` → 123456.78
`date`	Calendar date	`'2018-04-25'`
`time`	Time of day	`'09:30:00'`
`timestamp`	Date and time	`'2018-04-25 10:29:01'`

char vs varchar (INTUITION)

When to use `char` vs `varchar`:

- `char(n)`: Fixed-length codes (country codes: DK, US)
- `varchar(n)`: Variable data (names, addresses)

A.4 Data Definition Language (DDL)

A.4.1 CREATE TABLE

CREATE TABLE Syntax (CRITICAL)

```
CREATE TABLE tablename (
    column1  datatype  [constraints],
    column2  datatype  [constraints],
    ...
```

```
    [table constraints]
);
```

Example with constraints:

```
CREATE TABLE instructor (
    ID          char(5) PRIMARY KEY,
    name        varchar(20) NOT NULL,
    dept_name   varchar(20) REFERENCES department,
    salary      numeric(8,2) DEFAULT 50000.00
);
```

A.4.2 Integrity Constraints

Integrity Constraints

Constraint	Meaning
`PRIMARY KEY`	Uniquely identifies each row. Implies NOT NULL + UNIQUE.
`FOREIGN KEY`	References primary key of another table.
`NOT NULL`	Column cannot contain NULL values.
`UNIQUE`	All values must be distinct (NULL allowed once).
`DEFAULT`	Provides default value if none specified.

Primary Key vs Foreign Key (KEY CONCEPT)

- **Primary Key**: Unique identifier for THIS table's rows
- **Foreign Key**: Reference ("pointer") to another table's primary key

PK Implies NOT NULL (REMEMBER)

PRIMARY KEY automatically implies NOT NULL + UNIQUE.

A.4.3 Modifying Tables

Table Modification Operations (CRITICAL)

Know the difference between these operations:

Operation	Syntax	Effect
`DROP TABLE`	`DROP TABLE t;`	Removes entire table and data
`DELETE`	`DELETE FROM t;`	Removes all rows (table structure remains)
`ALTER TABLE`	`ALTER TABLE t ADD col type;`	Modifies table structure
`UPDATE`	`UPDATE t SET col=val;`	Modifies data values

`ALTER` changes table *structure*; `UPDATE` changes *data values*.

A.5 Basic Query Structure

A.5.1 SELECT-FROM-WHERE

Fundamental SQL Query Pattern (CRITICAL)

```
SELECT A1, A2, ..., An     -- What columns to return (projection)
FROM r1, r2, ..., rm       -- What tables to query
WHERE P;                   -- Filter condition (selection)
```

SQL to Relational Algebra Mapping (KEY CONCEPT)

- `SELECT` → Projection (Π)
- `WHERE` → Selection (σ)
- `FROM` multiple tables → Cartesian Product ($\times$)

```
SELECT A1, A2 FROM r WHERE P
```

is equivalent to: $\Pi_{A_1,A_2}(\sigma_P(r))$

A.5.2 The SELECT Clause

SELECT Examples with Input/Output

Input: Student

ID	name	age
1	Ali	20
2	Bob	22
3	Cat	20

Query: `SELECT name FROM Student;`
Plain English: Return all names from the Student table.
Output:

name
Ali
Bob
Cat

Query: `SELECT name, age FROM Student;`
Plain English: Return name and age columns from all students.
Output:

name	age
Ali	20
Bob	22
Cat	20

Query: `SELECT ID, age + 1 AS next_year FROM Student;`
Plain English: Return ID and age plus 1 (renamed as next_year) for each student.
Output:

ID	next_year
1	21
2	23
3	21

A.5.3 DISTINCT vs ALL

Duplicate Handling (CRITICAL)

SQL does NOT remove duplicates by default (unlike RA). Use `DISTINCT` to remove duplicates.

DISTINCT vs ALL Example

Input: Student

ID	name	age
1	Ali	20
2	Bob	22
3	Cat	20

Query: `SELECT age FROM Student;`
Plain English: Return all age values, including duplicates (SQL default behavior).
Output:

age
20
22
20

Note: age 20 appears twice!

Query: `SELECT DISTINCT age FROM Student;`
Plain English: Return unique age values only, removing all duplicates.
Output:

age
20
22

Now age 20 appears only once.

Sets vs Multisets (INTUITION)

Relations in RA are sets (no duplicates), but SQL allows duplicates for efficiency. Use `DISTINCT` when you want set behavior.

A.5.4 The WHERE Clause

WHERE Clause Operators

Comparison operators: `=`, `<>`, `<`, `<=`, `>`, `>=`
Logical operators: `AND`, `OR`, `NOT`
BETWEEN operator: Inclusive range check.
Boolean columns: `WHERE issenior` is equivalent to `WHERE issenior = true`

WHERE Clause Examples

Input: Student

ID	name	age
1	Ali	20
2	Bob	22
3	Cat	20
4	Dan	25

Query: `SELECT * FROM Student WHERE age = 20;`
Plain English: Return all columns for students whose age is exactly 20.
Output:

ID	name	age
1	Ali	20
3	Cat	20

Query: `SELECT name FROM Student WHERE age > 20 AND ID < 4;`
Plain English: Return names of students older than 20 AND with ID less than 4.
Output:

name
Bob

Query: `SELECT name FROM Student WHERE age BETWEEN 20 AND 22;`
Plain English: Return names of students whose age is between 20 and 22 (inclusive).
Output:

name
Ali
Bob
Cat

Note: BETWEEN is inclusive ($20 \leq \text{age} \leq 22$).

A.5.5 NULL Values

NULL Handling (CRITICAL)

NULL represents unknown or missing values.

```
WHERE dept_name IS NULL        -- Correct way to check NULL
WHERE dept_name IS NOT NULL    -- Check for non-NULL
```

Never Use

NEVER use `= NULL` or `<> NULL` — these don't work! Any comparison with NULL returns UNKNOWN.

A.6 Multi-Table Queries

A.6.1 Cartesian Product and Joins

Cartesian Product

Multiple tables in `FROM` create a Cartesian product (every pair of rows).

Cartesian Product Example

Input: Student

sID	name
1	Ali
2	Bob

Input: Course

cID
C1
C2

Query: `SELECT * FROM Student, Course;`
Plain English: Cartesian product: pair every student row with every course row.
Output: ($2 \times 2 = 4$ rows)

sID	name	cID
1	Ali	C1
1	Ali	C2
2	Bob	C1
2	Bob	C2

Meaningful Joins (CRITICAL)

Use `WHERE` to filter the Cartesian product into a meaningful join.

Join with WHERE Example

Input: Student

sID	name
1	Ali
2	Bob

Input: Enrolled

sID	course
1	Math
1	DB

Query: `SELECT name, course FROM Student, Enrolled WHERE Student.sID = Enrolled.sID;`
Plain English: Join Student and Enrolled on matching sID, return name and course.
Output:

name	course
Ali	Math
Ali	DB

Bob is excluded (no matching enrollment).

Join Conditions (KEY CONCEPT)

For n tables, you typically need $n - 1$ join conditions.

A.6.2 Table Aliases (AS)

Using Table Aliases

Input: instructor

ID	name	salary
10	Ali	50000
20	Bob	60000

Input: teaches

ID	course_id
10	CS101
10	CS202
20	MA101

Query:

```
SELECT i.name, t.course_id
FROM instructor AS i, teaches AS t
WHERE i.ID = t.ID;
```

Plain English: Join instructor (aliased as i) and teaches (aliased as t) on matching ID, return name and course_id.

Output:

name	course_id
Ali	CS101
Ali	CS202
Bob	MA101

Aliases `i` and `t` make the query shorter and clearer.

Using Column Aliases (Renaming Output)

Input: instructor

ID	name	salary
10	Ali	50000
20	Bob	60000

Query:

```
SELECT name AS instructor_name,
       salary / 12 AS monthly_salary
FROM instructor;
```

Plain English: Return name (renamed to instructor_name) and salary divided by 12 (renamed to monthly_salary).

Output:

instructor_name	monthly_salary
Ali	4166.67
Bob	5000.00

Column aliases rename output columns — useful for computed columns and clarity.

AS is Optional (REMEMBER)

The `AS` keyword is optional for both table and column aliases: `instructor i` and `salary / 12 monthly_salary` both work.

A.7 String Operations

A.7.1 LIKE Pattern Matching

LIKE Wildcards (CRITICAL)

Use when you see: "starts with", "ends with", "contains", "starting with"

Wildcards for pattern matching:

- % : Matches ANY substring (including empty string)
- _ : Matches exactly ONE character

Pattern	Matches
`'Intro%'`	Strings starting with "Intro"
`'%son'`	Strings ending with "son"
`'%Comp%'`	Strings containing "Comp"
`'___'`	Exactly 3 characters
`'___%'`	At least 3 characters

LIKE Example

```
SELECT name FROM instructor
WHERE name LIKE '%dar%';   -- Names containing "dar"
```

A.7.2 String Functions

String Functions

- `UPPER(s)`: Convert to uppercase
- `LOWER(s)`: Convert to lowercase
- `TRIM(s)`: Remove trailing spaces

A.8 ORDER BY Clause

ORDER BY Syntax

```
SELECT name, salary FROM instructor
ORDER BY salary DESC;              -- Descending order

SELECT name, salary FROM instructor
ORDER BY salary DESC, name ASC;    -- Multiple columns
```

Default Order (REMEMBER)

`ASC` (ascending) is the default. `ORDER BY` only affects display order, not the data.

A.9 Set Operations

Union Compatibility (CRITICAL)

Set operations require union-compatible queries (same number of columns, compatible types).

Operation	Meaning	RA Equivalent
`UNION`	Combines results, removes duplicates	$\cup$
`INTERSECT`	Common results only	$\cap$
`EXCEPT`	In first but not second	$-$

Set Operations Example

Input: MathStudents

name
Ali
Bob

Input: DBStudents

name
Bob
Cat

UNION: `(SELECT name FROM MathStudents) UNION (SELECT name FROM DBStudents);`
Plain English: All students in Math OR DB, duplicates removed.

name
Ali
Bob
Cat

Bob appears once (duplicates removed).
INTERSECT: `(SELECT name FROM MathStudents) INTERSECT (SELECT name FROM DBStudents);`
Plain English: Only students who are in BOTH Math AND DB.

name
Bob

EXCEPT: `(SELECT name FROM MathStudents) EXCEPT (SELECT name FROM DBStudents);`
Plain English: Students in Math but NOT in DB.

name
Ali

In Math but NOT in DB.

Keeping Duplicates (REMEMBER)

`UNION ALL`, `INTERSECT ALL`, `EXCEPT ALL` keep duplicates.

A.10 Aggregate Functions

Aggregate Functions (CRITICAL)

Operate on a set of values and return a single value:

Function	Description
`COUNT(*)`	Count all rows (including NULLs)
`COUNT(col)`	Count non-NULL values
`COUNT(DISTINCT col)`	Count distinct non-NULL values
`SUM(col)`	Sum of values (ignores NULLs)
`AVG(col)`	Average (ignores NULLs)
`MIN(col)`	Minimum value
`MAX(col)`	Maximum value

Aggregate Examples with Input/Output

Input: Student

ID	name	age
1	Ali	20
2	Bob	22
3	Cat	20
4	Dan	25

Query: `SELECT COUNT(*) FROM Student;`
Plain English: Count total number of rows in Student table.

count
4

Query: `SELECT COUNT(DISTINCT age) FROM Student;`

Plain English: Count the number of unique age values.

count
3

Only 3 distinct ages: 20, 22, 25.
Query: `SELECT AVG(age), MIN(age), MAX(age) FROM Student;`
Plain English: Compute average, minimum, and maximum age across all students.

avg	min	max
21.75	20	25

Query: `SELECT SUM(age) FROM Student WHERE age > 20;`
Plain English: Sum ages of students who are older than 20.

sum
47

Sum of $22 + 25 = 47$.

NULL Handling (REMEMBER)

NULL handling: `COUNT(*)` includes NULLs; all others ignore NULLs.

A.11 GROUP BY and HAVING

A.11.1 GROUP BY Clause

GROUP BY (CRITICAL)

Use when you see: "for each", "per", "in each", "number of X in Y"
Creates groups based on column values, then applies aggregates to **each group separately**. Each group produces exactly ONE output row.

GROUP BY: Step-by-Step Visualization

Input: Student

ID	dept	age
1	CS	20
2	CS	22
3	Math	21
4	Math	23

Query: `SELECT dept, COUNT(*), AVG(age) FROM Student GROUP BY dept;`
Plain English: For each department, count students and compute average age.
Step 1: Form Groups (rows with same `dept` grouped together)

Group	ID	dept	age
Group 1	1	CS	20
	2	CS	22
Group 2	3	Math	21
	4	Math	23

Step 2: Apply Aggregates to Each Group

dept	COUNT(*)	AVG(age)
CS	2	$(20 + 22)/2 = 21$
Math	2	$(21 + 23)/2 = 22$

Output: One row per group!

dept	count	avg
CS	2	21
Math	2	22

GROUP BY Rule (KEY CONCEPT)

Every non-aggregated column in SELECT **must** appear in GROUP BY!
WRONG: `SELECT dept, name, AVG(age) ... GROUP BY dept;`
Why? Each group has multiple `name` values — which one to show?

A.11.2 HAVING Clause

HAVING Clause (CRITICAL)

Use when you see: "groups having more than", "larger than N", "at least N"
Filters **groups** after aggregation. Use HAVING when the condition involves an aggregate function.

HAVING Example

Input: Student

ID	dept	age
1	CS	20
2	CS	22
3	Math	21
4	Math	23
5	Art	19

Query: `SELECT dept, COUNT(*) FROM Student GROUP BY dept HAVING COUNT(*) > 1;`
Plain English: Count students per department, but only show departments with more than 1 student.
After GROUP BY: (3 groups formed)

dept	count	passes HAVING?
CS	2	✓ $(2 > 1)$
Math	2	✓ $(2 > 1)$
Art	1	× $(1 \not> 1)$

Output: Art filtered out by HAVING

dept	count
CS	2
Math	2

A.11.3 WHERE vs HAVING: Key Difference

WHERE vs HAVING (CRITICAL)

	WHERE	HAVING
Filters	Individual rows	Groups
When	Before grouping	After grouping
Can use aggregates?	NO	YES

WHERE vs HAVING: Different Results!

Input: Student

ID	dept	age
1	CS	18
2	CS	22
3	Math	20
4	Math	24

Query with WHERE:

```
SELECT dept, AVG(age) FROM Student
WHERE age >= 20 GROUP BY dept;
```

Plain English: First filter to students aged 20+, then group by dept and compute average.
Step 1: WHERE removes rows with age $< 20 \rightarrow$ CS(18) removed
Step 2: GROUP BY on remaining rows

dept	avg	calculation
CS	22	only age 22 remains
Math	22	$(20 + 24)/2$

Query with HAVING:

```
SELECT dept, AVG(age) FROM Student
GROUP BY dept HAVING AVG(age) >= 20;
```

Plain English: Group by dept, compute average, then keep only groups with average >= 20.
Step 1: GROUP BY all rows (no filtering yet)

dept	avg	calculation
CS	20	$(18 + 22)/2$
Math	22	$(20 + 24)/2$

Step 2: HAVING filters: both ≥ 20, so both kept
Key Insight: CS average is **20** with HAVING but **22** with WHERE!

WHERE vs HAVING (INTUITION)

WHERE filters *individual ingredients* before cooking.
HAVING filters *finished dishes* after cooking.
Simple Rule: Condition uses aggregate (COUNT, SUM, AVG...)? → HAVING. Otherwise → WHERE.

A.12 Query Execution Order

Logical Execution Order (CRITICAL)

The logical execution order of SQL clauses:
1. **FROM**: Identify tables (Cartesian product)
2. **WHERE**: Filter rows
3. **GROUP BY**: Form groups
4. **HAVING**: Filter groups
5. **SELECT**: Compute output columns
6. **ORDER BY**: Sort results

Logical vs Physical (REMEMBER)

This is the *logical* order. Database optimizers may execute differently for performance.

A.13 Nested Subqueries

A.13.1 IN and NOT IN

IN and NOT IN Examples

Input: Student

sID	name
1	Ali
2	Bob
3	Cat

Input: Enrolled

sID	course
1	Math
1	DB

Query (IN): `SELECT name FROM Student WHERE sID IN (SELECT sID FROM Enrolled);`
Plain English: Return names of students whose sID appears in the Enrolled table.
Output:

name
Ali

Only Ali has enrollments.
Query (NOT IN): `SELECT name FROM Student WHERE sID NOT IN (SELECT sID FROM Enrolled);`
Plain English: Return names of students who have NO enrollments.
Output:

name
Bob
Cat

Students with NO enrollments.

NOT IN Pattern (CRITICAL)

Use when you see: "do not", "does not", "that don't have", "not in"
The `NOT IN` pattern is essential for "find entities that don't have X" queries.

A.13.2 SOME/ANY and ALL

SOME and ALL Examples

Input: Student

ID	name	age
1	Ali	20
2	Bob	22
3	Cat	25

Input: Senior (students over 21)

age
22
25

Query (SOME): `SELECT name FROM Student WHERE age > SOME (SELECT age FROM Senior);`
Plain English: Return students whose age is greater than at least one senior's age.
Output:

name
Cat

Cat (25) > 22 (at least one Senior). Bob (22) is NOT > 22.
Query (ALL): `SELECT name FROM Student WHERE age > ALL (SELECT age FROM Senior);`
Plain English: Return students whose age is greater than ALL seniors' ages.
Output:

name
(empty)

No one is older than ALL seniors (max is 25, which is not > 25).

A.13.3 EXISTS and NOT EXISTS

EXISTS Example

```
-- Departments with at least one instructor
SELECT dept_name FROM department AS d
WHERE EXISTS (SELECT * FROM instructor AS i
              WHERE i.dept_name = d.dept_name);
```

EXISTS Semantics (REMEMBER)

EXISTS checks if the subquery returns at least one row.

A.14 Common SQL Mistakes

Common SQL Mistakes

1. **Non-aggregated column not in GROUP BY**:

```
SELECT dept_name, name, AVG(salary) ...  -- WRONG!
GROUP BY dept_name;
```

2. **Using aggregate in WHERE instead of HAVING**:

```
WHERE COUNT(*) > 2   -- WRONG! Use HAVING
```

3. **Using = NULL instead of IS NULL**:

```
WHERE dept_name = NULL   -- WRONG! Always returns UNKNOWN
```

4. **Forgetting join conditions**:

```
SELECT * FROM t1, t2;   -- Creates Cartesian product!
```

A.15 SQL Quick Reference

SQL Quick Reference

Task	Pattern
Filter rows	`WHERE condition`
Join tables	`FROM t1, t2 WHERE t1.key = t2.key`
Remove duplicates	`SELECT DISTINCT ...`
Pattern match	`WHERE col LIKE '%pattern%'`
NULL check	`WHERE col IS NULL / IS NOT NULL`
Count rows	`SELECT COUNT(*) FROM ...`
Group and filter	`GROUP BY col HAVING condition`
Set union	`(query1) UNION (query2)`
Not in set	`WHERE col NOT IN (subquery)`
Existence check	`WHERE EXISTS (subquery)`

A.16 SQL Advanced (L04)

Overview

This section covers advanced SQL topics: NULL handling, data modifications, join variants, advanced subqueries, views, and integrity constraints.

A.16.1 NULL Values

NULL Semantics (CRITICAL)

NULL represents "unknown" or "does not exist" - it is NOT the same as zero or empty string.

A.16.1.1 Three-Valued Logic

Three-Valued Logic (REMEMBER)

SQL uses **three-valued logic**: TRUE, FALSE, and UNKNOWN.
Any comparison with NULL yields UNKNOWN:

- `5 < NULL` → UNKNOWN
- `NULL = NULL` → UNKNOWN (not TRUE!)
- `NULL <> NULL` → UNKNOWN

AND	T	U	F
T	T	U	F
U	U	U	F
F	F	F	F

OR	T	U	F
T	T	T	T
U	T	U	U
F	T	U	F

NOT	
T	F
U	U
F	T

Understanding UNKNOWN (INTUITION)

Think of UNKNOWN as "maybe" - AND with maybe is maybe (unless definitely false), OR with maybe is maybe (unless definitely true).

A.16.1.2 Testing for NULL

IS NULL Syntax (CRITICAL)

Use `IS NULL` and `IS NOT NULL` - never use `= NULL`!

```
-- CORRECT:
SELECT * FROM instructor WHERE salary IS NULL;

-- WRONG (always returns empty):
SELECT * FROM instructor WHERE salary = NULL;
```

A.16.1.3 NULL in Aggregates

NULL in Aggregates

- **SUM, AVG, MIN, MAX**: Ignore NULL values
- **COUNT(*)**: Counts all rows including those with NULLs
- **COUNT(column)**: Counts non-NULL values only

NULL in Aggregates Example

Input: Student

ID	name	age
1	Ali	20
2	Bob	NULL
3	Cat	22

Query: `SELECT COUNT(*), COUNT(age), AVG(age) FROM Student;`
Plain English: Count all rows, count non-NULL ages, and compute average age (ignoring NULLs).
Output:

count(*)	count(age)	avg(age)
3	2	21

`COUNT(*)` counts all 3 rows. `COUNT(age)` counts only 2 non-NULL ages. `AVG(age)` = (20+22)/2 = 21 (ignores NULL).

All NULLs (REMEMBER)

If all values are NULL, aggregates return NULL (except COUNT which returns 0).

A.16.2 Data Modification

A.16.2.1 INSERT

INSERT Syntax

```
INSERT INTO table VALUES (val1, val2, ...);              -- All columns
INSERT INTO table (col1, col2) VALUES (val1, val2);      -- Specific columns
INSERT INTO table SELECT ... FROM ...;                   -- From query
```

INSERT Example

Before: Student

ID	name	age
1	Ali	20
2	Bob	22

Query: `INSERT INTO Student VALUES (3, 'Cat', 21);`
Plain English: Insert a new row with ID=3, name='Cat', age=21.
After: Student

ID	name	age
1	Ali	20
2	Bob	22
3	Cat	21

A.16.2.2 DELETE

DELETE Syntax

```
DELETE FROM table;                     -- Delete ALL rows
DELETE FROM table WHERE condition;     -- Delete specific rows
```

DELETE Example

Before: Student

ID	name	age
1	Ali	20
2	Bob	22
3	Cat	20

Query: `DELETE FROM Student WHERE age = 20;`
Plain English: Remove all rows where age equals 20.
After: Student

ID	name	age
2	Bob	22

Ali and Cat (both age 20) are deleted.

DELETE vs DROP (REMEMBER)

DELETE removes entire rows. To remove a column, use ALTER TABLE DROP COLUMN.

A.16.2.3 UPDATE

UPDATE Syntax

```
UPDATE table SET col = value;                      -- Update all rows
UPDATE table SET col = value WHERE condition;      -- Update specific rows
```

UPDATE Examples

Before: Student

ID	name	age
1	Ali	20
2	Bob	22
3	Cat	20

Query: `UPDATE Student SET age = age + 1 WHERE age = 20;`

Plain English: Add 1 to age for all students currently aged 20.
After: Student

ID	name	age
1	Ali	21
2	Bob	22
3	Cat	21

Ali and Cat now have age 21 (was 20).

UPDATE with CASE Example

Before: Student

ID	name	age
1	Ali	20
2	Bob	22

Query:

```
UPDATE Student SET age = CASE
    WHEN age < 21 THEN age + 2
    ELSE age + 1
END;
```

Plain English: For students under 21, add 2 to age; for others, add 1.
After: Student

ID	name	age
1	Ali	22
2	Bob	23

Ali (under 21): +2. Bob (21+): +1.

CASE Expression (CRITICAL)

CASE expressions are essential for conditional updates:

```
CASE WHEN cond1 THEN val1 WHEN cond2 THEN val2 ELSE default END
```

A.16.3 Join Expressions

Join Types (KEY CONCEPT)

SQL provides explicit JOIN syntax beyond the basic comma-separated FROM clause.

Join Type	Description
`INNER JOIN`	Only matching tuples from both relations (default)
`LEFT OUTER JOIN`	All tuples from left + matching from right (NULL-padded)
`RIGHT OUTER JOIN`	All tuples from right + matching from left (NULL-padded)
`FULL OUTER JOIN`	All tuples from both, NULL-padded where no match
`NATURAL JOIN`	Equi-join on all common attribute names
`CROSS JOIN`	Cartesian product

A.16.3.1 Join Conditions

Join Syntax Examples

```
-- NATURAL JOIN (implicit equi-join on common attributes):
SELECT * FROM student NATURAL JOIN takes;
```

```
-- JOIN with ON clause (explicit condition):
SELECT * FROM student JOIN takes ON student.ID = takes.ID;

-- JOIN with USING clause (equi-join on specified columns):
SELECT * FROM student JOIN takes USING (ID);
```

NATURAL JOIN Warning (INTUITION)

NATURAL JOIN is convenient but dangerous - it joins on ALL common attribute names, which may not be what you want. Prefer explicit ON or USING.

A.16.3.2 Outer Join Examples

Outer Join Examples with Input/Output

Input: Student

sID	name
1	Ali
2	Bob

Input: Enrolled

sID	course
1	Math
3	DB

INNER JOIN: `SELECT * FROM Student JOIN Enrolled ON Student.sID = Enrolled.sID;`
Plain English: Return only rows where sID matches in both tables.

sID	name	course
1	Ali	Math

Only matching rows.
LEFT OUTER JOIN: `SELECT * FROM Student LEFT JOIN Enrolled ON Student.sID = Enrolled.sID;`
Plain English: Keep ALL students; if no enrollment match, fill with NULL.

sID	name	course
1	Ali	Math
2	Bob	NULL

ALL students kept; Bob has NULL for course.
RIGHT OUTER JOIN: `SELECT * FROM Student RIGHT JOIN Enrolled ON Student.sID = Enrolled.sID;`
Plain English: Keep ALL enrollments; if no student match, fill with NULL.

sID	name	course
1	Ali	Math
3	NULL	DB

ALL enrollments kept; sID=3 has NULL for name.
FULL OUTER JOIN: `SELECT * FROM Student FULL JOIN Enrolled ON Student.sID = Enrolled.sID;`
Plain English: Keep ALL rows from both tables; fill missing values with NULL.

sID	name	course
1	Ali	Math
2	Bob	NULL
3	NULL	DB

ALL from both sides.

Outer Join Semantics (CRITICAL)

Outer joins preserve tuples that have no match - the missing attributes are filled with NULL.

A.16.4 Set Membership (IN, NOT IN)

IN and NOT IN (CRITICAL)

Trigger words: "in at least one", "member of", "not in", "excluding", "does not"

IN	Value exists in set/subquery result
NOT IN	Value does NOT exist in set/subquery result

IN Example

Input: Student **Input: Enrolled**

sID	name
1	Ali
2	Bob

sID
1

```
SELECT name FROM Student WHERE sID IN (SELECT sID FROM Enrolled);
```

Plain English: Find students whose sID appears in the Enrolled table.

Output:

name
Ali

sID 1 is in Enrolled.

NOT IN Example

```
SELECT name FROM Student WHERE sID NOT IN (SELECT sID FROM Enrolled);
```

Plain English: Find students whose sID does NOT appear in Enrolled.

Output:

name
Bob

sID 2 is NOT in Enrolled.

Equivalences (REMEMBER)

`x IN (subquery)` is the same as `x = SOME (subquery)`

Why? Both ask: "Is x equal to at least one value in the set?"

`x NOT IN (subquery)` is the same as `x <> ALL (subquery)`

Why? Both ask: "Is x different from every value in the set?"

A.16.5 Set Comparison (SOME, ALL)

SOME and ALL (CRITICAL)

Trigger words: "more than at least one" → SOME | "more than all" → ALL

`> SOME`	Greater than at least one	$= \exists$
`> ALL`	Greater than every value	$= \forall$

SOME Example

Input: Student

sID	name	age
1	Ali	20
2	Bob	25
3	Cat	30

```
SELECT name FROM Student WHERE age > SOME (SELECT age FROM Student);
```

Plain English: Find students whose age is greater than at least one other student's age.

Subquery returns: {20, 25, 30}

Output:

name
Bob
Cat

Bob (25>20), Cat (30>20). Ali excluded (20 not > any).

ALL Example

```
SELECT name FROM Student WHERE age > ALL (SELECT age FROM Student WHERE age < 30);
```

Plain English: Find students whose age is greater than ALL students under 30.
Subquery returns: {20, 25}
Output:

name
Cat

Cat (30 > 25 AND 30 > 20).

Empty Subquery Behavior (CRITICAL)

Expression	Meaning	Result	Why
`> ALL (empty)`	Greater than every	TRUE	Nothing to be greater than
`> SOME (empty)`	Greater than at least one	FALSE	No value exists to compare
`IN (empty)`	Exists in set	FALSE	Can't be in an empty set
`NOT IN (empty)`	Not in set	TRUE	Not in empty = always true

A.16.6 Advanced Subqueries

A.16.6.1 Scalar Subqueries

Scalar Subqueries

A **scalar subquery** returns exactly one value (one row, one column).

Scalar Subquery Example

Input: Dept

dName
CS
Math

Input: Instructor

name	dName
Ali	CS
Bob	CS
Cat	Math

```
SELECT dName, (SELECT COUNT(*) FROM Instructor WHERE Instructor.dName = Dept.dName)
AS cnt FROM Dept;
```

Output: CS: 2, Math: 1

Scalar Subquery Error (REMEMBER)

If scalar subquery returns more than one value → runtime error.

A.16.6.2 Correlated Subqueries

Correlated Subqueries

A **correlated subquery** references attributes from the outer query. Inner re-evaluated for each outer row.

Correlated Subquery Example

Input: Student

name	dept	age
Ali	CS	20
Bob	CS	25
Cat	Math	22

```
SELECT name FROM Student S WHERE age > (SELECT AVG(age) FROM Student WHERE dept =
```

```
S.dept);
```
Plain English: Students older than their department's average.
Output: Bob (25 > avg(CS)=22.5)

A.16.6.3 EXISTS and NOT EXISTS

EXISTS Semantics (CRITICAL)

EXISTS tests whether a subquery returns any tuples (non-empty).

EXISTS and NOT EXISTS Examples

Input: Student

sID	name
1	Ali
2	Bob
3	Cat

Input: Enrolled

sID	course
1	Math
1	DB

Query (EXISTS):

```
SELECT name FROM Student S
WHERE EXISTS (SELECT * FROM Enrolled E WHERE E.sID = S.sID);
```

Plain English: Return names of students who have at least one enrollment.
Output:

name
Ali

Only Ali has enrollments (EXISTS returns true for Ali).
Query (NOT EXISTS):

```
SELECT name FROM Student S
WHERE NOT EXISTS (SELECT * FROM Enrolled E WHERE E.sID = S.sID);
```

Plain English: Return names of students who have NO enrollments at all.
Output:

name
Bob
Cat

Bob and Cat have NO enrollments (NOT EXISTS returns true).

NOT EXISTS (REMEMBER)

NOT EXISTS is true when the subquery returns **empty** result.

A.16.6.4 Derived Relations (Subqueries in FROM)

Derived Relations

```
-- Subquery in FROM clause (derived table):
SELECT dept_name, avg_salary
FROM (SELECT dept_name, AVG(salary) AS avg_salary
      FROM instructor
      GROUP BY dept_name) AS dept_avg
WHERE avg_salary > 42000;
```

A.16.6.5 WITH Clause (Common Table Expressions)

WITH Clause (CRITICAL)

The WITH clause defines temporary named relations for use within a query.

```
WITH dept_total (dept_name, value) AS
    (SELECT dept_name, SUM(salary)
     FROM instructor
     GROUP BY dept_name),
dept_total_avg(value) AS
    (SELECT AVG(value) FROM dept_total)
SELECT dept_name
FROM dept_total, dept_total_avg
WHERE dept_total.value > dept_total_avg.value;
```

CTE Benefits (INTUITION)

WITH (CTE) makes complex queries more readable by breaking them into named steps. The temporary relations exist only for that query.

A.16.7 Views

View Concept (KEY CONCEPT)

A **view** is a virtual relation defined by a query. It does not store data - it's computed on demand.

A.16.7.1 Creating Views

View Example with Input/Output

Input: Student

ID	name	age
1	Ali	20
2	Bob	22
3	Cat	21

Create View: `CREATE VIEW YoungStudents AS SELECT ID, name FROM Student WHERE age < 22;`
Plain English: Create a virtual table showing ID and name of students under 22.
View: YoungStudents (virtual table)

ID	name
1	Ali
3	Cat

Query the view: `SELECT name FROM YoungStudents;`
Plain English: Get names from the virtual table (students under 22).
Output:

name
Ali
Cat

The view hides the `age` column and filters to age < 22.

A.16.7.2 View Expansion

View Expansion

When you query a view, the DBMS replaces the view name with its definition:

```
-- Query:
SELECT name FROM faculty WHERE dept_name = 'Biology';

-- Becomes:
```

```
SELECT name FROM (SELECT ID, name, dept_name FROM instructor)
WHERE dept_name = 'Biology';
```

A.16.7.3 Updatable Views

Updatable View Conditions (REMEMBER)

A view is **updatable** only if:

- FROM has exactly one database relation
- SELECT contains only attribute names (no expressions, aggregates, DISTINCT)
- No GROUP BY or HAVING
- Attributes not in SELECT can be NULL or have defaults

View Update Anomaly

```
-- This view IS updatable:
CREATE VIEW history_instructors AS
SELECT * FROM instructor WHERE dept_name = 'History';

-- Insert through view:
INSERT INTO history_instructors VALUES ('25566', 'Brown', 'Biology', 100000);
-- Warning: This tuple won't appear in the view (dept_name != 'History')!
```

A.16.7.4 Materialized Views

Materialized Views

A **materialized view** actually stores the query result. It must be refreshed when underlying data changes.

```
CREATE MATERIALIZED VIEW dept_avg_salary AS
SELECT dept_name, AVG(salary) AS avg_salary
FROM instructor
GROUP BY dept_name;
```

Regular vs Materialized (INTUITION)

Regular views: computed every time (always current, may be slow). Materialized views: stored (fast, but may be stale).

A.16.8 Integrity Constraints

Integrity Constraints (KEY CONCEPT)

Integrity constraints ensure data validity. They are checked on every modification.

A.16.8.1 All Constraint Types Overview

Constraint Types Summary (CRITICAL)

Constraint	Purpose	Key Points
`PRIMARY KEY`	Uniquely identifies each row	Unique + NOT NULL; one per table
`NOT NULL`	Prevents NULL values	Column must always have a value
`UNIQUE`	Ensures all values are distinct	Allows NULL (unlike PRIMARY KEY)
`CHECK`	Validates against a condition	Can reference same-row columns only
`FOREIGN KEY`	Links to another table's key	Ensures referential integrity
`DEFAULT`	Provides automatic value	Used when INSERT omits the column

Constraint Syntax Quick Reference

Constraint	Syntax Example
PRIMARY KEY	`ID INT PRIMARY KEY` or `PRIMARY KEY (ID)`
NOT NULL	`name VARCHAR(20) NOT NULL`
UNIQUE	`email VARCHAR(50) UNIQUE` or `UNIQUE (col1, col2)`
CHECK	`CHECK (age >= 0)` or `CHECK (salary > 0)`
FOREIGN KEY	`FOREIGN KEY (dept) REFERENCES Dept(dName)`
DEFAULT	`status VARCHAR(10) DEFAULT 'active'`

A.16.8.2 Single-Relation Constraints

Complete Constraint Example

```
CREATE TABLE student (
    ID VARCHAR(5) PRIMARY KEY,            -- Unique + NOT NULL
    name VARCHAR(20) NOT NULL,            -- Cannot be NULL
    email VARCHAR(50) UNIQUE,             -- Unique but can be NULL
    dept_name VARCHAR(20) DEFAULT 'Undeclared',  -- Default value
    tot_cred NUMERIC(3,0) CHECK (tot_cred >= 0), -- Must be >= 0
    UNIQUE (name, dept_name)              -- Composite unique constraint
);
```

PRIMARY KEY vs UNIQUE (CRITICAL)

Property	PRIMARY KEY	UNIQUE
Allows NULL?	No	Yes
How many per table?	Only 1	Multiple allowed
Creates index?	Yes (clustered)	Yes (non-clustered)

A.16.8.3 Referential Integrity (Foreign Keys)

Foreign Key Constraint (CRITICAL)

A **foreign key** ensures that values in one table exist in another table (referential integrity).

```
CREATE TABLE course (
    course_id VARCHAR(8) PRIMARY KEY,
    title VARCHAR(50),
    dept_name VARCHAR(20),
    credits NUMERIC(2,0) CHECK (credits > 0),
    FOREIGN KEY (dept_name) REFERENCES department(dept_name)
);
```

Foreign Key Rules (CRITICAL)

A foreign key must reference a column (or set of columns) that is **unique** in the referenced table:

- Can reference `PRIMARY KEY` ✓
- Can reference `UNIQUE` constraint (including composite) ✓
- **Cannot** reference a column that allows duplicates ×

Additional rules:

- FK column can be NULL (unless also declared NOT NULL)
- Data types must match between FK and referenced column

A.16.8.4 Referential Actions

Referential Actions (CRITICAL)

What happens when a referenced tuple is deleted or updated?

Action	Effect	Use When
`CASCADE`	Delete/update referencing tuples automatically	Child data meaningless without parent
`SET NULL`	Set FK attributes to NULL	Child can exist independently
`SET DEFAULT`	Set FK to default value	Has meaningful default
`RESTRICT`	Reject the operation immediately	Protect important references
`NO ACTION`	Reject (checked at end of statement)	Default behavior

Referential Action Syntax

```
CREATE TABLE Student (
    ID INT PRIMARY KEY,
    name VARCHAR(20) NOT NULL,
    dName VARCHAR(20),
    FOREIGN KEY (dName) REFERENCES Dept(dName)
        ON DELETE CASCADE      -- Delete student if dept deleted
        ON UPDATE SET NULL     -- Set NULL if dept name changes
);
```

Referential Actions Example

Dept (referenced)

dName
CS
Math

Student (references Dept with ON DELETE CASCADE)

ID	name	dName
1	Ali	CS
2	Bob	CS
3	Cat	Math

Action: `DELETE FROM Dept WHERE dName = 'CS';`
Plain English: Delete the CS department (CASCADE will also delete students in CS).
After — Dept:

dName
Math

After — Student: (CASCADE deletes referencing rows)

ID	name	dName
3	Cat	Math

Ali and Bob are automatically deleted because they referenced CS.
With `SET NULL`: Ali and Bob would remain but with dName = NULL.
With `RESTRICT`: The delete would be rejected (CS still referenced).

A.16.8.5 Complex CHECK Constraints

Complex CHECK

```
-- Check constraint with subquery (not widely supported):
CHECK (time_slot_id IN (SELECT time_slot_id FROM time_slot))
```

A.16.8.6 Assertions

Assertions

An **assertion** is a condition that must always be true for the entire database:

```
CREATE ASSERTION credits_earned_constraint CHECK
(NOT EXISTS (
    SELECT ID FROM student
    WHERE tot_cred <> (SELECT COALESCE(SUM(credits), 0)
                       FROM takes NATURAL JOIN course
                       WHERE takes.ID = student.ID
                         AND grade IS NOT NULL AND grade <> 'F')
));
```

Assertions Rarely Used (REMEMBER)

Assertions are expensive to check and not widely supported. Most systems use triggers instead.

A.16.9 Transactions

Transaction Concept (CRITICAL)

A **transaction** consists of a sequence of query and/or update statements and is a "unit" of work. Transactions ensure that either ALL operations complete successfully, or NONE of them take effect.

A.16.9.1 Transaction Control Statements

Transaction Commands (REMEMBER)

Command	Syntax	Effect
BEGIN	begin;	Start a new transaction
COMMIT	commit;	End transaction and make all changes **permanent** and **visible** to others
ROLLBACK	rollback;	Reset transaction and **discard** all changes made

Transaction Example

```
-- Transfer $100 from account A to account B
begin;
    UPDATE account SET balance = balance - 100 WHERE acct_id = 'A';
    UPDATE account SET balance = balance + 100 WHERE acct_id = 'B';
commit;   -- Both updates become permanent

-- If something goes wrong:
begin;
    UPDATE account SET balance = balance - 100 WHERE acct_id = 'A';
    -- Error occurs here!
rollback; -- Undo the debit from A, database unchanged
```

Transaction Intuition (INTUITION)

Think of a transaction like a "save point" in a video game. You can make many changes, and at the end:

- `commit` = Save the game (changes are permanent)
- `rollback` = Load previous save (discard all changes)

Atomicity (CRITICAL)

Transactions are **atomic**: they either complete entirely or have no effect at all. This is crucial for maintaining database consistency (e.g., money transfers, inventory updates).

A.16.10 Common Patterns from Exercises

A.16.10.1 Pattern: Finding Maximum/Minimum

Finding Max/Min

```
-- Find instructor(s) with highest salary:
SELECT name, salary FROM instructor
WHERE salary = (SELECT MAX(salary) FROM instructor);

-- Alternative using ALL:
SELECT name, salary FROM instructor
WHERE salary >= ALL (SELECT salary FROM instructor);
```

A.16.10.2 Pattern: Counting with Conditions

Counting with Conditions

```
-- Count students per department who have taken > 3 courses:
SELECT dept_name, COUNT(DISTINCT ID)
FROM student NATURAL JOIN takes
GROUP BY dept_name
HAVING COUNT(DISTINCT course_id) > 3;
```

A.16.10.3 Pattern: NOT EXISTS for "All" Queries

NOT EXISTS Pattern

```
-- Find students who have taken ALL courses in Biology dept:
SELECT S.ID, S.name
FROM student AS S
WHERE NOT EXISTS (
    SELECT course_id FROM course WHERE dept_name = 'Biology'
    EXCEPT
    SELECT course_id FROM takes WHERE takes.ID = S.ID
);
```

Double Negation (INTUITION)

"For all X, P(X)" becomes "NOT EXISTS X such that NOT P(X)". This is the double negation pattern.

A.16.10.4 Pattern: Self-Join for Comparisons

Self-Join Pattern

```
-- Find pairs of instructors in same department:
SELECT DISTINCT i1.name, i2.name
FROM instructor i1, instructor i2
WHERE i1.dept_name = i2.dept_name AND i1.ID < i2.ID;
```

A.16.11 Summary: SQL Advanced Quick Reference

SQL Advanced Quick Reference

Topic	Key Points
NULL	Three-valued logic; use IS NULL, not = NULL
INSERT	VALUES for single tuple; SELECT for bulk insert
UPDATE	Use CASE for conditional updates
DELETE	Removes rows, not columns
Joins	INNER (default), LEFT/RIGHT/FULL OUTER, NATURAL
EXISTS	Tests for non-empty subquery result
WITH (CTE)	Named temporary relations for complex queries
Views	Virtual relations; may be updatable
Constraints	PRIMARY KEY, FOREIGN KEY, CHECK, UNIQUE, NOT NULL
Transactions	begin/commit/rollback; atomic unit of work

A.17 Entity-Relationship Model (L05)

ER Model Purpose

The ER model is used for **conceptual design** - modeling what data exists and how things relate, before thinking about tables.

A.17.1 ER Model Basics

A.17.1.1 Core Concepts

Core ER Concepts (CRITICAL)

Three fundamental ER concepts:

Concept	Definition	ER Notation
Entity	Object that exists and is distinguishable	–
Entity Type	Description/schema of entities	Rectangle
Entity Set	Collection of all entities of same type	–
Attribute	Property of an entity	Oval
Relationship	Association among entities	Diamond

Entity Type vs Entity Set (INTUITION)

Think of Entity Type as a class definition (e.g., `instructor(ID, name, salary)`). Entity Set is all instances. Entity is one specific instance.

A.17.1.2 Finding Entities and Relationships

Design Heuristics (KEY CONCEPT)

When designing from requirements:

- **Nouns** often become entities (customer, order, product)
- **Verbs** often become relationships (owns, buys, teaches)

A.17.1.3 Attribute Types

Attribute Types

Type	Description	ER Notation
Simple	Single value (name)	Single oval
Composite	Sub-attributes (address → street, city)	Oval with sub-ovals
Single-valued	One value (ID)	Single oval
Multi-valued	Multiple values (phone numbers)	Double oval
Stored	Directly stored (birthdate)	Regular oval
Derived	Computed from others (age from birthdate)	Dashed oval

A.17.1.4 Primary Keys

Primary Keys in ER

- Set of attributes that **uniquely identify** an entity
- **Underlined** in ER diagram
- No two entities can have the same key values

A.17.2 Relationships

A.17.2.1 Relationship Basics

Relationship Concepts

- **Relationship**: Association among entities
- **Relationship Set**: Set of relationships of same type
- **Degree**: Number of participating entity sets
  - Binary (degree 2): Most common
  - Ternary (degree 3): Rare, more complex

A.17.2.2 Roles (Recursive Relationships)

Recursive Relationship

When the same entity type participates multiple times, use **roles**:

```
course --prereq--> course
  |                    |
course_id        prereq_id (roles)
```

A.17.2.3 Relationship Attributes

Relationship Attributes

Relationships can have their own attributes:

- `takes(semester, year, grade)` - attributes of the relationship
- These belong to the relationship, not the entities

A.17.3 Mapping Cardinality

Cardinality (CRITICAL)

Cardinality specifies how many entities can be associated via a relationship.

A.17.3.1 Four Types of Binary Relationships

Binary Relationship Types

Type	Meaning	Example
One-to-One (1:1)	Each A with at most one B, vice versa	person ↔ passport
One-to-Many (1:N)	One A with many B; each B with one A	instructor → students
Many-to-One (N:1)	Each A with one B; one B with many A	students → department
Many-to-Many (M:N)	Many A with many B, vice versa	students ↔ courses

A.17.3.2 Three Notations (Know All Three!)

Notation Knowledge (KEY CONCEPT)

The exam requires you to read and convert between all three notations.

Graphical Notation

1. Graphical Notation:

- Arrow (→) points to the "one" side
- Line (−) indicates "many" side

```
instructor -----> student    (one instructor, many students = 1:N)
instructor <----> student    (one-to-one)
instructor <----- student    (many-to-one from instructor view)
instructor ------ student    (many-to-many)
```

Chen Notation

2. Chen Notation:

Numbers on lines: 1, N, M

```
instructor --1--<>--N-- student    (one-to-many)
instructor --1--<>--1-- student    (one-to-one)
instructor --N--<>--M-- student    (many-to-many)
```

[Min, Max] Notation (CRITICAL)

3. [Min, Max] Notation:

Most precise notation - specifies both minimum and maximum.

[min, max]	Meaning	Participation
$[0, 1]$	Optional, at most one	Partial
$[1, 1]$	Exactly one (mandatory)	Total
$[0, *]$	Optional, any number	Partial
$[1, *]$	At least one (mandatory)	Total

```
instructor --[0,*]--<>--[1,1]-- student
(instructor: 0 or more students; student: exactly 1 instructor)
```

Reading [Min,Max] (REMEMBER)

Reading direction: The number NEAR an entity tells how many of THAT entity can participate.

A.17.4 Participation Constraints

Participation Types (KEY CONCEPT)

Participation specifies whether EVERY entity must participate.

Type	Meaning	ER Notation	[min,max]
Total	Every entity MUST participate	Double line (=)	min ≥ 1
Partial	Some entities MAY NOT participate	Single line (−)	min $= 0$

Participation Example

```
instructor ---<>===  student
             ^   ^
             |   |
         partial  total

- Instructor: partial (some may not supervise anyone)
- Student: total (every student MUST have a supervisor)
```

Total vs Partial (INTUITION)

Total = "must have". Partial = "may have".

A.17.5 Keys for Relationship Sets

Relationship Keys (CRITICAL)

The primary key of a relationship depends on its cardinality.

Cardinality	Relationship Key	Why
M:N	$\text{key}(E_1) \cup \text{key}(E_2)$	Need both to identify
1:N	key(N-side entity)	N-side determines relationship
N:1	key(N-side entity)	Same as above
1:1	key(either entity)	Either side works

Relationship Key Logic (INTUITION)

In 1:N, each "many" entity connects to exactly one "one" entity, so knowing the "many" entity uniquely determines the relationship. In M:N, you need both sides.

A.17.6 Weak Entity Types

Weak Entity (CRITICAL)

A weak entity cannot be uniquely identified by its own attributes alone.

A.17.6.1 Definition

Weak Entity Properties

- Existence **depends on** another entity (the **identifying entity**)
- Needs identifying entity's key + its own **discriminator**

Example: `section` depends on `course`

- `section(sec_id, semester, year)` - NOT unique alone
- Need `course.course_id` to identify the section
- Full key: `(course_id, sec_id, semester, year)`

A.17.6.2 Discriminator (Partial Key)

Discriminator

- Attributes that distinguish weak entities related to the **same owner**
- Underlined with **dashed line** in ER diagram
- NOT a full key by itself

A.17.6.3 ER Notation for Weak Entities

Weak Entity Notation

Element	Notation
Weak entity type	Double rectangle
Identifying relationship	Double diamond
Discriminator	Dashed underline

```
[course] ===<>=== [[section]]
   |        1:N        |
course_id          sec_id (discriminator)
```

A.17.6.4 Rules for Weak Entities

Weak Entity Rules (REMEMBER)

Three key rules:

1. Always **total participation** in identifying relationship
2. Always on the "**many**" side (1:N)
3. Strong (identifying) entity on "one" side

Primary Key of Weak Entity:

$$\text{PK(weak)} = \text{PK(identifying_entity)} + \text{discriminator(weak)}$$

A.17.7 Specialization (ISA Hierarchy)

Specialization Concept (KEY CONCEPT)

Specialization creates sub-groupings with inherited attributes.

A.17.7.1 Concept

ISA Hierarchy

- Top-down design: entity set → subclasses
- Lower-level entities **inherit all attributes** from higher level
- Notation: Triangle with "ISA" label

```
       [person]          <- superclass (ID, name)
          |
         ISA
        /   \
[employee]  [student]    <- subclasses
(+ salary)  (+ tot_cred)
```

A.17.7.2 Attribute Inheritance

Inheritance (CRITICAL)

Subclass has ALL attributes of superclass PLUS its own:

- `person`: (ID, name)
- `employee`: (ID, name, salary) - inherits + adds
- `student`: (ID, name, tot_cred) - inherits + adds

A.17.7.3 Overlapping vs Disjoint

Overlapping vs Disjoint

Type	Meaning	Example
Overlapping	Can belong to MULTIPLE subclasses	Person can be employee AND student
Disjoint	AT MOST ONE subclass	Employee is instructor OR secretary

Determining Overlap (REMEMBER)

Ask: "Can an entity be both X and Y?" If yes → overlapping. If no → disjoint.

A.17.7.4 Total vs Partial Specialization

Total vs Partial Specialization

Type	Meaning
Total	Every superclass entity MUST be in some subclass
Partial	Some superclass entities may not be in any subclass

A.17.8 ER Design Process

A.17.8.1 Steps to Design an ER Diagram

ER Design Steps

1. **Identify entity types** from nouns in requirements
2. **Identify relationships** from verbs
3. **Determine cardinality** from "one", "many", "zero or more"
4. **Add attributes** (2-3 meaningful attributes per entity)
5. **Add participation constraints** from "must", "may"
6. **Add min-max notation** matching cardinalities
7. **Identify weak entities** if any depend on others

A.17.8.2 Design Goals

Design Goals

- **Avoid redundancy**: Don't repeat information
- **Avoid incompleteness**: Model all needed aspects
- **Minimal design**: Don't add unnecessary ID attributes

A.17.9 Exercise Pattern: Car Insurance Example

Car Insurance Example

From L05 exercises (HIGH priority):
Entities: customer, car, policy, accident, payment
Relationships and Cardinalities:

Relationship	Cardinality	Min-Max
owns (customer-car)	1:N	customer[1,*], car[1,1]
covers (policy-car)	1:N	policy[1,*], car[1,1]
participated (car-accident)	M:N	car[0,*], accident[1,*]
paying (policy-payment)	1:N	policy[1,*], payment[1,1]

Weak Entity Pattern (REMEMBER)

payment can be modeled as weak entity (depends on policy) with paymentNo as discriminator.

A.17.10 Summary: ER Notation Quick Reference

ER Notation Quick Reference

Element	Symbol
Entity type	Rectangle
Weak entity type	Double rectangle
Attribute	Oval
Multi-valued attribute	Double oval
Derived attribute	Dashed oval
Primary key	Underlined
Discriminator	Dashed underline
Relationship	Diamond
Identifying relationship	Double diamond
Total participation	Double line
Partial participation	Single line

A.18 ER to Relational Mapping (L06)

Overview

This section covers how to convert an ER diagram into a relational schema using the 7-step mapping algorithm.

A.18.1 The 7-Step Mapping Algorithm

Memorize These Steps (CRITICAL)

Memorize these 7 steps - they define the systematic conversion process.

Step	What to Map	Key Action
M1	Regular (strong) entity types	Create relation with all attributes
M2	Weak entity types	Include FK to owner + discriminator in PK
M3	Binary M:N relationships	Create new relation with both PKs
M4	Binary 1:N relationships	Add FK to the N-side
M5	Binary 1:1 relationships	Add FK to one side (prefer total)
M6	Complex relationships	Recursive: self-FK; Ternary: all PKs
M7	Multivalued attributes	Create separate relation

A.18.2 M1: Mapping Regular Entity Types

M1: Regular Entity Types

Rules:

1. Create ONE relation for each entity type
2. Include all SIMPLE attributes as columns
3. Primary key = entity type's primary key

Example:

```
ER: [course] with attributes (course_id, title, credits)

Relation: course(course_id, title, credits)
                 ^^^^^^^^^ PK
```

A.18.3 M2: Mapping Weak Entity Types

M2: Weak Entity Types (CRITICAL)

Weak entity's PK = owner's PK + discriminator.

Rules:

1. Create ONE relation for the weak entity
2. Include all attributes + PK from identifying entity as FK
3. Primary key = FK (owner's PK) + discriminator

Weak Entity Example

```
ER: course ===<>== [[section]] (section is weak)

course(course_id, title, credits)
section(course_id, sec_id, semester, year)
        ^^^^^^^^^ ^^^^^^^
        FK (from  discriminator
        course)

PK of section = (course_id, sec_id, semester, year)
```

Weak Entity Key (REMEMBER)

The weak entity's relation ALWAYS includes the identifying entity's PK as both FK and part of its own PK.

A.18.4 M3: Mapping Binary M:N Relationships

M3: M:N Relationships (CRITICAL)

M:N relationships ALWAYS require a new relation.

Rules:

1. Create NEW relation for the relationship
2. Include PKs from BOTH entity types as FKs
3. Primary key = UNION of both PKs
4. Include any relationship attributes

M:N Relationship Example

```
ER: instructor --N--<>--M-- student (supervise)

instructor(i_id, name, salary)
student(s_id, name)
supervise(i_id, s_id, meeting)
          ^^^^ ^^^^  ^^^^^^^
          FK   FK    rel attr

PK of supervise = (i_id, s_id)
```

Why Separate Table? (INTUITION)

Why separate table? In M:N, one student can have many instructors AND one instructor can have many students. You cannot store this in either entity's table without duplication.

A.18.5 M4: Mapping Binary 1:N Relationships

M4: 1:N Relationships (KEY CONCEPT)

Extend the N-side (many side) with a foreign key.

Two Options:

Option 1 (separate table): Like M:N, but PK = N-side PK only

Option 2 (PREFERRED): Add FK to the "many" side relation

- FK references "one" side's PK
- Include relationship attributes in N-side

1:N Relationship Example (Option 2)

```
ER: instructor --1--<>--N-- student (supervise)

instructor(i_id, name, salary)
student(s_id, name, i_id, meeting)
                    ^^^^  ^^^^^^^
                    FK    rel attr (added to N-side)
```

Why Extend N-side? (INTUITION)

Why extend N-side? Each "many" entity connects to exactly ONE "one" entity. So each N-side row needs exactly one FK value - no duplication. Extending the 1-side would require multiple FK values per row (impossible).

A.18.6 M5: Mapping Binary 1:1 Relationships

M5: 1:1 Relationships

Rule: Extend ONE side's relation with FK to the other side.

Which side to extend?

- Prefer the **TOTAL participation** side (avoids NULLs)
- If both total: either side works (or merge into one relation)
- If both partial: either side works (will have some NULLs)

1:1 Relationship Example

```
ER: instructor --1--<>--1== student (supervise)
               partial    total

instructor(i_id, name, salary)
student(s_id, name, i_id, meeting)
                    ^^^^
                    FK on total side (no NULLs)
```

Total Participation (REMEMBER)

Total participation = every entity MUST have a relationship (no NULLs). Partial = some may not (NULLs needed).

A.18.7 M6: Complex Relationships

A.18.7.1 Recursive (Self-Referencing) Relationships

Recursive Relationships

Same entity type on both sides - use role names for FKs.

M:N Recursive

M:N Recursive:

```
ER: course --N--<>--M-- course (prereq)
         after       pre

course(course_id, title, credits)
prereq(after_course_id, pre_course_id)
       ^^^^^^^^^^^^^^^  ^^^^^^^^^^^^^
       FK (renamed)     FK (renamed)

PK = (after_course_id, pre_course_id)
```

1:N Recursive

1:N Recursive:

```
ER: course --1--<>--N-- course (prereq)
         pre      after

course(course_id, title, credits, pre_course_id)
                                  ^^^^^^^^^^^^^
                                  FK to itself
```

Rename FKs (REMEMBER)

Rename FKs to distinguish roles!

A.18.7.2 Ternary Relationships

Ternary Relationships

N:M:P (all many):

```
Create relation with ALL THREE PKs
R(PK1, PK2, PK3)
Primary key = (PK1, PK2, PK3)
```

N:M:1 (one side is "one"):

```
Create relation, but PK is only the "many" sides
R(PK1, PK2, PK3)
Primary key = (PK1, PK2)  -- only N and M sides!
PK3 is FK only, not part of PK
```

A.18.8 M7: Complex Attributes

A.18.8.1 Multivalued Attributes

Multivalued Attributes (CRITICAL)

Create a SEPARATE relation!

```
ER: employee with multivalued attribute phone

employee(e_id, name, salary)
phone(e_id, phone_number)
      ^^^^ ^^^^^^^^^^^^
      FK   part of PK

PK of phone = (e_id, phone_number)
```

Why Separate Table? (INTUITION)

Multiple values cannot be stored in one column (violates 1NF). Each value becomes a separate row.

A.18.8.2 Composite Attributes

Composite Attributes

Include component attributes directly (flatten):

```
ER: employee has composite address(street, city, state)

employee(e_id, name, salary, street, city, state)
                             ^^^^^^^^^^^^^^^^^^^^^
                             flattened composite
```

A.18.8.3 Derived Attributes

Derived Attributes

IGNORE during mapping - can add as view later:

```
ER: employee has derived attribute age (from birthday)

employee(e_id, name, salary, birthday)
                             ^^^^^^^^ stored
-- age NOT stored, computed when needed
```

A.18.9 Mapping Specialization (ISA)

ISA Mapping (CRITICAL)

The relational model does NOT support inheritance. Four alternatives exist.

Example ISA

```
       person(ID, name, gender)
              |
             ISA
            /   \
    employee     student
    (salary)    (credits)
```

A.18.9.1 Alternative 1: Main Classes

Alternative 1: Main Classes

Each entity stored in its most specific class only:

```
person(ID, name, gender)            -- only pure persons
employee(ID, name, gender, salary)  -- includes inherited
student(ID, name, gender, credits)  -- includes inherited
```

Properties: No redundancy. Query all persons requires UNION.

A.18.9.2 Alternative 2: Partitioning

Alternative 2: Partitioning

Attributes split across tables, key duplicated:

```
person(ID, name, gender)       -- ALL persons
employee(ID, salary)           -- only employee-specific (FK to person)
student(ID, credits)           -- only student-specific (FK to person)
```

Properties: No attribute redundancy (except PK). Join needed for full info.

A.18.9.3 Alternative 3: Full Redundancy

Alternative 3: Full Redundancy

Subclass tables include ALL inherited attributes, parent also has all:

```
person(ID, name, gender)            -- ALL persons
employee(ID, name, gender, salary)  -- duplicates person data
student(ID, name, gender, credits)  -- duplicates person data
```

Properties: Redundant. Update anomalies. No joins needed.

A.18.9.4 Alternative 4: Single Relation

Alternative 4: Single Relation

Everything in one table with type discriminator:

```
person(ID, name, gender, type, salary, credits)
```

Properties: Many NULLs. Simple queries. Wastes space.

A.18.9.5 Choosing an Alternative

Choosing ISA Alternative

Alternative	Good When	Bad When
1: Main classes	Entities rarely change type	Query all persons often
2: Partitioning	Avoid redundancy priority	Need full entity often
3: Full redundancy	Read performance critical	Updates frequent
4: Single relation	Few subclass attributes	Many NULLs expected

A.18.10 Exercise Pattern: Car Insurance Mapping

Car Insurance Mapping

From L06 exercises (HIGH priority):

Final Relations:

```
-- Strong entities
customer(customerID, address, name)
policy(policyID)
accident(reportID, date, place)

-- Entity with merged 1:N relationships
car(vehicleIDnumber, model, licenseNo, customerID, policyID)
                                       ^^^^^^^^^^ ^^^^^^^^^
                                       FK         FK

-- Weak entity
payment(policyID, paymentNo, receivedOn, amount, dueDate)
        ^^^^^^^^  ^^^^^^^^^
        FK        discriminator (together = PK)

-- M:N relationship
participated(vehicleIDnumber, reportID)
             ^^^^^^^^^^^^^^^^ ^^^^^^^^
             FK               FK (together = PK)
```

A.18.11 Summary: Quick Mapping Reference

Quick Mapping Reference

ER Element	Action	PK Rule
Strong entity	Direct mapping	Entity's PK
Weak entity	Include owner's PK as FK	Owner PK + discriminator
M:N relationship	New relation	Both entity PKs
1:N relationship	FK in N-side	–
1:1 relationship	FK in total side	–
Multivalued attr	Separate relation	Owner PK + attr value
Composite attr	Flatten (include components)	–
Derived attr	Ignore (compute later)	–

Exam Pattern (REMEMBER)

The exam pattern: "Given this ER diagram, create the relational schema." Apply M1-M7 systematically.

A.19 Normalization (L07)

Overview

Normalization eliminates redundancy by decomposing relations based on functional dependencies (FDs).

A.19.1 Why Normalize?

Data Anomalies

Poor design causes **data anomalies**:

- **Redundancy**: Same info repeated (dept building stored for every instructor)
- **Update Anomaly**: Inconsistent updates (change building for one instructor, not others)
- **Deletion Anomaly**: Unintended data loss (delete last instructor → lose dept info)

A.19.2 Functional Dependencies

Functional Dependency (CRITICAL)

An FD $X \to Y$ means: if two tuples agree on X, they must agree on Y.

FD Intuition (INTUITION)

X values **uniquely determine** Y values. Like a function: $f(X) = Y$.

Examples:

- ID $\to$ name (ID determines name)
- dept_name $\to$ building (department determines building)

FDs are NOT Symmetric

$A \to B$ does **NOT** imply $B \to A$.

Example: StudentID $\to$ Name, but Name $\not\to$ StudentID (multiple students can have same name).

A.19.2.1 Trivial vs Non-trivial

Trivial vs Non-trivial FDs

Type	Definition	Example
Trivial	$Y \subseteq X$ (RHS is subset of LHS)	$\{A, B\} \to \{B\}$
Non-trivial	$Y \not\subseteq X$	ID $\to$ name

Trivial FDs (REMEMBER)

Trivial FDs always hold (tautology) - no need to prove!

A.19.2.2 Armstrong's Axioms

Armstrong's Axioms (CRITICAL)

These three rules derive ALL valid FDs from a given set.

Axiom	Rule	Example
Reflexivity	If $Y \subseteq X$, then $X \to Y$	$\{A, B, C\} \to \{B, C\}$
Augmentation	If $X \to Y$, then $XZ \to YZ$	If $A \to B$, then $AC \to BC$
Transitivity	If $X \to Y$ and $Y \to Z$, then $X \to Z$	If $A \to B$, $B \to C$, then $A \to C$

A.19.2.3 Derived Rules

Derived Rules

Rule	Formula
Union	If $X \to Y$ and $X \to Z$, then $X \to YZ$
Decomposition	If $X \to YZ$, then $X \to Y$ and $X \to Z$
Pseudotransitivity	If $X \to Y$ and $YZ \to W$, then $XZ \to W$

A.19.3 Deriving FDs with Armstrong's Axioms

When to Use Each Axiom (KEY CONCEPT)

Understanding **when** to apply each rule is the key to FD derivations.

Rule	You Have	You Want	Result
Reflexivity	Any set X	Subset of X	$X \to Y$ where $Y \subseteq X$
Augmentation	$X \to Y$	Add attrs to both sides	$XZ \to YZ$
Transitivity	$X \to Y$ and $Y \to Z$	Chain through Y	$X \to Z$
Decomposition	$X \to YZ$	Only part of RHS	$X \to Y$ and $X \to Z$
Union	$X \to Y$ and $X \to Z$	Combine same LHS	$X \to YZ$
Pseudo-trans	$X \to Y$ and $YW \to Z$	Bridge with extra attrs	$XW \to Z$

Derivation Strategy (CRITICAL)

Step-by-step approach for any FD derivation:

Step	Action	Ask Yourself
1	Identify target FD: $X \to Y$	What do I need to prove?
2	Find FDs with X on LHS	Do I already have $X \to$ something?
3	Find FDs that produce Y	What FDs have Y (or parts of Y) on RHS?
4	Build a chain	Can I connect X to Y through intermediate FDs?
5	Apply axioms	Which rule transforms what I have into what I need?
6	Document each step	State the rule used at every step!

Proving a New Rule: Pseudo-Transitivity II

Goal: Prove that $\alpha \to \beta\gamma \wedge \beta \to \delta \Rightarrow \alpha \to \beta\delta\gamma$

Analysis: We start with $\alpha \to \beta\gamma$ and $\beta \to \delta$, want to end with $\alpha \to \beta\delta\gamma$.

Step	From	Derive	Rule
1	$\alpha \to \beta\gamma$	$\alpha \to \beta$ and $\alpha \to \gamma$	Decomposition
2	$\alpha \to \beta$, $\beta \to \delta$	$\alpha \to \delta$	Transitivity
3	$\alpha \to \delta$, $\alpha \to \gamma$	$\alpha \to \delta\gamma$	Union
4	$\alpha \to \beta$, $\alpha \to \delta\gamma$	$\alpha \to \beta\delta\gamma$	Union

Key insight: Break down $\to$ extend via transitivity $\to$ recombine via union.

Worked Example: Deriving FDs Step-by-Step

Given $R = (A, B, C, D, E, F, G)$ with FD set F:

#	FD	#	FD
f_1	$A \to BC$	f_4	$CA \to F$
f_2	$E \to CGA$	f_5	$F \to CDGB$
f_3	$C \to DA$	f_6	$CD \to BF$

Task 1: Derive $C \to D$

Strategy: Look for FDs with C on LHS. Found $f_3 : C \to DA$. Just need to extract D.

Step	Input FD(s)	Output FD	Rule Applied
1	$C \to DA$ (from f_3)	$C \to D$	Decomposition

Task 2: Derive $F \to A$

Strategy: F appears on LHS in f_5. A appears on RHS of f_3. Build a chain: $F \to ... \to A$.

Step	Input FD(s)	Output FD	Rule Applied
1	$F \to CDGB$ (from f_5)	$F \to C$	Decomposition
2	$F \to C$, $C \to DA$ (from f_3)	$F \to DA$	Transitivity
3	$F \to DA$	$F \to A$	Decomposition

Chain visualization: $F \xrightarrow{f_5} C \xrightarrow{f_3} DA \xrightarrow{\text{decomp}} A$

Task 3: Derive $E \to ABCDEFG$ (Prove E is a candidate key)

Strategy: Start with $E \to CGA$, progressively add attributes using Pseudo-Trans II.

Step	Input FD(s)	Output FD	Rule Applied
1	$E \to CGA$, $C \to DA$	$E \to ACDG$	Pseudo-Trans II
2	$E \to ACDG$, $A \to BC$	$E \to ABCDG$	Pseudo-Trans II
3	$E \to ABCDG$, $CD \to BF$	$E \to ABCDFG$	Pseudo-Trans II
4	$E \to ABCDFG$	$EE \to EABCDFG$	Augmentation
5	$EE = E$	$E \to ABCDEFG$	Simplification

Attribute accumulation:

After Step	Attributes Determined by E
Start (f_2)	$\{E, C, G, A\}$
Step 1	$\{E, A, C, D, G\}$
Step 2	$\{E, A, B, C, D, G\}$
Step 3	$\{E, A, B, C, D, F, G\}$
Step 5	$\{A, B, C, D, E, F, G\} = R$

Conclusion: $E^+ = R$, so E is a candidate key!

Rule Selection Guide (REMEMBER)

Which rule do I need? Use this decision table:

Situation	Rule to Use	Pattern
RHS has too many attributes	Decomposition	$X \to YZ \Rightarrow X \to Y$
Two FDs have same LHS	Union	$X \to Y, X \to Z \Rightarrow X \to YZ$
RHS of one = LHS of another	Transitivity	$X \to Y, Y \to Z \Rightarrow X \to Z$
Need to add attr to LHS	Augmentation	$X \to Y \Rightarrow XW \to YW$
Partial RHS match to another LHS	Pseudo-Trans	$X \to YZ, Y \to W \Rightarrow X \to YZW$
Need trivial FD (subset)	Reflexivity	$XY \to X$ always holds

Common Mistakes in FD Derivations

Mistake	Wrong	Why It's Wrong
Assuming symmetry	$A \to B \Rightarrow B \to A$	FDs are NOT bidirectional
Wrong transitivity	$A \to B, C \to D \Rightarrow A \to D$	RHS must equal LHS exactly
Skipping steps	"Obviously $A \to D$"	Must show each rule applied
Inventing FDs	Using FDs not in F or derived	Can only use given or derived FDs
Wrong decomposition	$AB \to C \Rightarrow A \to C$	Can only split RHS, not LHS

Exam Tip: Derivation Questions (REMEMBER)

Format your answer as a table:

Step	Derivation	Rule
1	...	...

This makes it easy for graders to follow and award partial credit!

A.19.4 Attribute Closure

Attribute Closure (CRITICAL)

The closure X^+ = all attributes functionally determined by X under F.

A.19.4.1 Closure Algorithm

Closure Algorithm

```
result := alpha
while (result changes) do
    for each FD beta -> gamma in F do
        if beta is subset of result then
            result := result UNION gamma
return result
```

A.19.4.2 Example

Closure Example

Given: $R = (A, B, C, G, H, I)$, $F = \{A \to B, A \to C, CG \to H, CG \to I, B \to H\}$

Compute $(AG)^+$:

1. result = AG
2. $A \to B$: A $\subseteq$ AG $\Rightarrow$ result = ABG

3. $A \to C$: A $\subseteq$ ABG $\Rightarrow$ result = ABCG
4. $CG \to H$: CG $\subseteq$ ABCG $\Rightarrow$ result = ABCGH
5. $CG \to I$: CG $\subseteq$ ABCGH $\Rightarrow$ result = ABCGHI
6. $(AG)^+ = ABCGHI$

A.19.4.3 Using Closure to Verify FDs

Verifying FDs

To check if $X \to Y$ holds: Compute X^+. If $Y \subseteq X^+$, then $X \to Y$ holds.

A.19.5 Keys

Key Definitions (KEY CONCEPT)

- **Superkey**: K where $K^+ = R$ (determines all attributes)
- **Candidate Key**: Minimal superkey (no proper subset is a superkey)
- **Primary Key**: One chosen candidate key

A.19.5.1 Prime vs Non-Prime Attributes

Prime and Non-Prime Attributes (CRITICAL)

- **Prime Attribute**: Any attribute that is part of **at least one candidate key**
- **Non-Prime Attribute**: Any attribute that is **NOT part of any candidate key**

Prime/Non-Prime Example 1: Composite Key

$R(A, B, C, D)$ with FDs: $AB \to C$, $AB \to D$
Candidate Key: AB

Attribute	Prime?	Reason
A		Part of key AB
B		Part of key AB
C		Not part of any key
D		Not part of any key

Prime/Non-Prime Example 2: Single-Attribute Key

$R(A, B, C, D, E, F)$ with FDs: $AB \to C$, $B \to A$, $ABC \to D$, $D \to EF$
Candidate Key: B (since $B^+ = \{B, A, C, D, E, F\}$)

Attribute	Prime?	Reason
B		B is THE candidate key
A, C, D, E, F		Not part of key B

Key insight: When the candidate key is a single attribute, ONLY that attribute is prime!

Prime/Non-Prime Example 3: Multiple Candidate Keys

$R(A, B, C)$ with FDs: $AB \to C$, $C \to A$
Candidate Keys: AB and BC (both can derive all attributes)

Attribute	Prime?	Reason
A		Part of key AB
B		Part of keys AB and BC
C		Part of key BC

All attributes are prime! This is why this relation satisfies 3NF even with $C \to A$.

A.19.5.2 Finding Candidate Keys

Finding Candidate Keys Algorithm

Algorithm:
1. Identify attributes that NEVER appear on RHS of any FD - these MUST be in every key
2. Compute closures starting with smallest attribute sets

3. If $X^+ = R$, X is a superkey
4. Check if any proper subset of X is also a superkey - if not, X is a candidate key

Key Finding Trick (REMEMBER)

Trick: Attributes never on RHS MUST be in every candidate key!

Finding Candidate Keys Example

$R = (A, B, C, D, E)$, $F = \{A \to BC, CD \to E, B \to D, E \to A\}$

Find candidate keys:

- $A^+ = ABCDE$ ✓ (superkey)
- $E^+ = EABCD = ABCDE$ ✓ (superkey)
- $(BC)^+ = BCDEA = ABCDE$ ✓ (superkey, BC is minimal)
- $(CD)^+ = CDEAB = ABCDE$ ✓ (superkey, CD is minimal)

Candidate Keys: A, E, BC, CD

A.19.6 Normal Forms

Normal Form Hierarchy (CRITICAL)

Normal form hierarchy: $1NF \supset 2NF \supset 3NF \supset BCNF$

Each higher form is stricter and eliminates more redundancy.

A.19.6.1 First Normal Form (1NF)

First Normal Form (1NF)

Rule: All attribute values must be **atomic** (single, indivisible values).

- No multi-valued attributes
- No composite attributes

1NF Violation

VIOLATES 1NF:

A	B
1	X, Y, Z
2	P, Q

Why: B contains multiple values in a single cell. This is NOT atomic.

1NF Satisfied

HOLDS 1NF:

A	B
1	X
1	Y
1	Z
2	P
2	Q

Why: Every cell contains exactly ONE value. All values are atomic.

A.19.6.2 Second Normal Form (2NF)

Second Normal Form (2NF) (CRITICAL)

Rule: Must be in 1NF + **No partial dependencies**.

Every non-prime attribute must depend on the **entire** candidate key, not just part of it.

What is a Partial Dependency?

A **partial dependency** occurs when a non-prime attribute depends on **only PART** of a composite key.

```
Key = AB (composite, two attributes)

AB -> C   [OK] C depends on FULL key
```

```
A -> C     [PARTIAL] C depends on PART of key
B -> D     [PARTIAL] D depends on PART of key
```

Plain English: If your key is AB, then every non-key attribute should need BOTH A and B to be determined. If C only needs A, that's a partial dependency.
Note: Partial dependencies can ONLY exist with composite keys (2+ attributes). If your key is a single attribute, partial dependencies are impossible!

2NF Violation

$R(A, B, C, D)$ with FDs: $AB \to C$, $AB \to D$, $A \to C$
Key: AB (composite)
Prime: A, B **Non-prime:** C, D

A	B	C	D
1	X	M	P
1	Y	M	Q
2	X	N	R

Why it violates:
- $A \to C$ means C depends only on A (part of key AB)
- This is a **partial dependency**
- Notice: C = M is repeated for A = 1 (redundancy!)

Fix by decomposition:
- $R_1(A, C)$ with FD: $A \to C$
- $R_2(A, B, D)$ with FD: $AB \to D$

2NF Satisfied

$R(A, B, C, D)$ with FDs: $AB \to C$, $AB \to D$
Key: AB

A	B	C	D
1	X	M	P
1	Y	N	Q
2	X	O	R

Why it holds:
- $AB \to C$: C depends on FULL key ✓
- $AB \to D$: D depends on FULL key ✓
- No attribute depends on just A or just B alone
- **No partial dependencies!**

A.19.6.3 Third Normal Form (3NF)

Third Normal Form (3NF) (CRITICAL)

Rule: Must be in 2NF + **No transitive dependencies**.
Formal Definition: For every non-trivial FD $X \to Y$, at least one must hold:
1. X is a superkey, OR
2. Every attribute in $(Y - X)$ is part of some candidate key (i.e., Y is prime)

What is a Transitive Dependency?

A **transitive dependency** occurs when a non-prime attribute depends on another non-prime attribute (creating a chain through non-key attributes).

```
Key = A

A -> B     [OK] B depends on key
B -> C     [TRANSITIVE] C depends on B, which is not a key
```

```
Chain: A -> B -> C (C is transitively dependent on A through B)
```

Plain English: If you can only find out C by first looking up B (and B isn't a key), that's a transitive dependency. C doesn't directly depend on the key.
Real Example:

```
Key = StudentID

StudentID -> DeptID       [Direct dependency on key]
DeptID -> DeptName        [TRANSITIVE! DeptName depends on DeptID]

Chain: StudentID -> DeptID -> DeptName
```

3NF Violation

$R(A, B, C)$ with FDs: $A \to B$, $B \to C$
Key: A
Prime: A **Non-prime:** B, C

A	B	C
1	X	M
2	X	M
3	Y	N

Why it violates:
- $B \to C$ where B is non-prime and C is non-prime
- This creates: $A \to B \to C$ (transitive chain)
- B is NOT a superkey
- C is NOT a prime attribute
- Notice: C = M is repeated because B = X is repeated (redundancy!)

Fix by decomposition:
- $R_1(A, B)$ with FD: $A \to B$
- $R_2(B, C)$ with FD: $B \to C$

3NF Satisfied

$R(A, B, C)$ with FDs: $A \to B$, $A \to C$
Key: A

A	B	C
1	X	M
2	Y	N
3	Z	O

Why it holds:
- $A \to B$: A is superkey ✓
- $A \to C$: A is superkey ✓
- No non-prime attribute determines another non-prime attribute
- **No transitive dependencies!**

3NF Satisfied - Prime Attribute on RHS

$R(A, B, C)$ with FDs: $AB \to C$, $C \to A$
Keys: AB, BC (both are candidate keys!)
Prime: A, B, C (all are part of some key!)

A	B	C
1	X	M
1	Y	M
2	X	N

Why it holds 3NF:
- $AB \to C$: AB is superkey ✓
- $C \to A$: A is a **prime attribute** (part of key AB) ✓
- Even though C is not a superkey, A is prime, so 3NF allows it!

A.19.6.4 BCNF (Boyce-Codd Normal Form)

BCNF (CRITICAL)

Rule: For **EVERY** non-trivial FD $X \to Y$:

$$X \text{ MUST be a superkey. No exceptions!}$$

Difference from 3NF: BCNF has NO exception for prime attributes on RHS.

BCNF Violation (but holds 3NF!)

$R(A, B, C)$ with FDs: $AB \to C$, $C \to A$
Keys: AB, BC
Prime: A, B, C

A	B	C
1	X	M
1	Y	M
2	X	N

Check each FD:

FD	3NF Check	BCNF Check
$AB \to C$	AB is superkey	AB is superkey
$C \to A$	A is prime attr	C is NOT superkey

Why it violates BCNF:
- $C \to A$, but C is not a superkey ($C^+ = \{C, A\}$, not all attributes)
- 3NF allows this because A is prime
- **BCNF does NOT allow any exception!**
- Notice: A = 1 is repeated because C = M appears twice

Fix by decomposition:
- $R_1(C, A)$ with FD: $C \to A$
- $R_2(B, C)$ with key: BC

BCNF Satisfied

$R(A, B, C)$ with FDs: $A \to B$, $A \to C$
Key: A

A	B	C
1	X	M
2	Y	N
3	Z	O

Check each FD:

FD	Is LHS superkey?	BCNF?
$A \to B$	A is the key	
$A \to C$	A is the key	

Why it holds: Every FD has a superkey on the left side. No exceptions, no violations!

A.19.6.5 3NF vs BCNF: The Key Difference

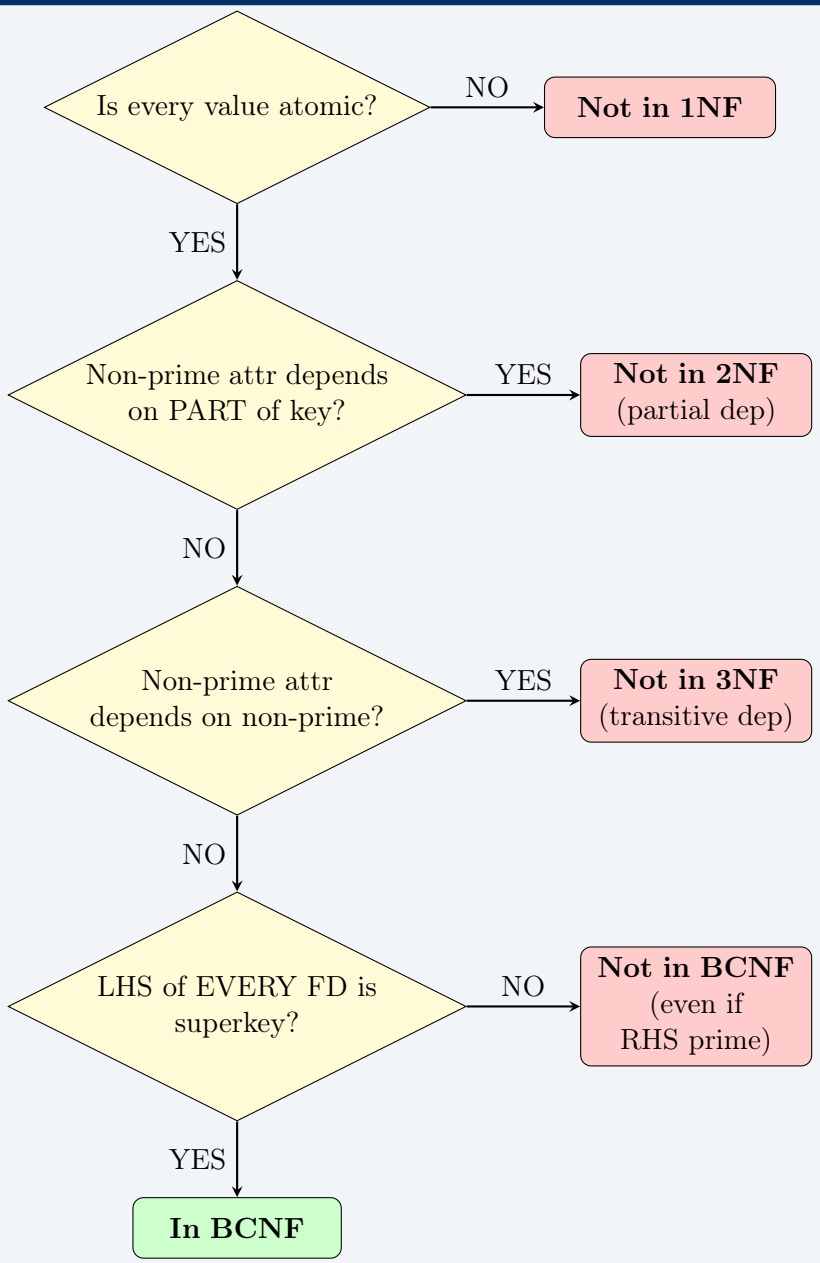
3NF vs BCNF Comparison (KEY CONCEPT)

Scenario for FD $X \to Y$	3NF	BCNF
X is superkey		
Y is prime attribute (X not superkey)		
X not superkey, Y not prime		

BCNF is stricter! It doesn't care if RHS is prime - it ONLY cares that LHS is a superkey.

A.19.6.6 Checking Normal Forms: Quick Decision Flowchart

Normal Form Decision Flowchart (CRITICAL)



Hierarchy Check for Single FD: $X \to Y$

1. **Is X a superkey?**
   - YES → Satisfies ALL (2NF, 3NF, BCNF)
   - NO → Continue to step 2
2. **Is Y a prime attribute?**
   - YES → Satisfies 3NF (but NOT BCNF)
   - NO → Continue to step 3
3. **Is X part of a composite key?**

- YES → Violates 2NF (partial dependency)
- NO → Violates 3NF (transitive dependency)

A.19.6.7 Red Flag Patterns

Spotting Violations - Red Flag Patterns

2NF Violation Pattern:

```
Key = AB (composite)
A -> C    <-- Part of key on LHS = PARTIAL DEPENDENCY
```

3NF Violation Pattern:

```
Key = A
B -> C    <-- Non-prime -> Non-prime = TRANSITIVE DEPENDENCY
```

BCNF Violation Pattern:

```
Keys = AB, BC
C -> A    <-- C is not superkey (even though A is prime!)
```

A.19.7 Decomposition

Decomposition

Decomposition splits a relation into smaller relations to eliminate redundancy.

A.19.7.1 Lossless Decomposition

Lossless Decomposition (CRITICAL)

A decomposition is **lossless** if we can reconstruct the original relation by joining.
Lossless Condition: The intersection must be a superkey for at least one of the decomposed relations.

$$(R_1 \cap R_2) \to (R_1 - R_2) \quad \text{OR} \quad (R_1 \cap R_2) \to (R_2 - R_1)$$

Lossless Intuition (INTUITION)

The common attributes between R_1 and R_2 must be able to uniquely identify tuples in at least one of them. Otherwise, joining creates spurious (fake) tuples.

Lossless Decomposition: 4-Step Method (KEY CONCEPT)

Given decomposition into R_1 and R_2:
Step 1: Attribute Distribution – Create a table showing which attributes are in each relation
Step 2: Find Intersection – $R_1 \cap R_2$ = common attributes
Step 3: Compute Closure of Intersection – Calculate $(R_1 \cap R_2)^+$
Step 4: Check Lossless Condition

- $R_1 - R_2$ = attributes only in R_1
- $R_2 - R_1$ = attributes only in R_2
- Does intersection closure contain $R_1 - R_2$? OR Does it contain $R_2 - R_1$?
- YES to either → **LOSSLESS**
- NO to both → **NOT LOSSLESS**

Lossless Decomposition Analysis

$R = (A, B, C, D, E, F)$ with FDs: $AB \to C$, $B \to A$, $ABC \to D$, $D \to EF$
Test: $R_1 = (A, B, C, D)$ and $R_2 = (A, B, E, F)$

Step 1: Attribute Distribution

	A	B	C	D	E	F
R_1	✓	✓	✓	✓	×	×
R_2	✓	✓	×	×	✓	✓

Step 2: Find Intersection

$R_1 \cap R_2 = \{A, B, C, D\} \cap \{A, B, E, F\} = \{\mathbf{A}, \mathbf{B}\}$

Step 3: Compute Closure of Intersection
$(AB)^+$:
- Start: $\{A, B\}$
- $AB \to C$: $\{A, B, C\}$
- $ABC \to D$: $\{A, B, C, D\}$
- $D \to EF$: $\{A, B, C, D, E, F\}$

$(\mathbf{AB})^+ = \{\mathbf{A}, \mathbf{B}, \mathbf{C}, \mathbf{D}, \mathbf{E}, \mathbf{F}\}$

Step 4: Check Lossless Condition
- $R_1 - R_2 = \{C, D\}$
- Does $(AB)^+$ contain $\{C, D\}$? **YES** – $(AB)^+$ contains C and D

Result: **LOSSLESS**

Not Lossless Decomposition Analysis

$R = (A, B, C, D, E, F)$ with FDs: $AB \to C$, $B \to A$, $ABC \to D$, $D \to EF$
Test: $R_1 = (A, B, C)$ and $R_2 = (A, D, E, F)$

Step 1: Attribute Distribution

	A	B	C	D	E	F
R_1	✓	✓	✓	×	×	×
R_2	✓	×	×	✓	✓	✓

Step 2: Find Intersection
$R_1 \cap R_2 = \{A, B, C\} \cap \{A, D, E, F\} = \{\mathbf{A}\}$

Step 3: Compute Closure of Intersection
A^+: No FD has A alone on the left side.
$\mathbf{A}^+ = \{\mathbf{A}\}$

Step 4: Check Lossless Condition
- $R_1 - R_2 = \{B, C\}$ – Does $A^+ = \{A\}$ contain $\{B, C\}$? **NO**
- $R_2 - R_1 = \{D, E, F\}$ – Does $A^+ = \{A\}$ contain $\{D, E, F\}$? **NO**

Result: **NOT LOSSLESS**

Another Lossless Example

$R = (A, B, C, D, E, F)$ with FDs: $AB \to C$, $B \to A$, $ABC \to D$, $D \to EF$
Test: $R_1 = (B, D, E)$ and $R_2 = (A, B, C, F)$

Step 1: Attribute Distribution

	A	B	C	D	E	F
R_1	×	✓	×	✓	✓	×
R_2	✓	✓	✓	×	×	✓

Step 2: Find Intersection
$R_1 \cap R_2 = \{B, D, E\} \cap \{A, B, C, F\} = \{\mathbf{B}\}$

Step 3: Compute Closure of Intersection
B^+:
- Start: $\{B\}$
- $B \to A$: $\{A, B\}$
- $AB \to C$: $\{A, B, C\}$
- $ABC \to D$: $\{A, B, C, D\}$
- $D \to EF$: $\{A, B, C, D, E, F\}$

$\mathbf{B}^+ = \{\mathbf{A}, \mathbf{B}, \mathbf{C}, \mathbf{D}, \mathbf{E}, \mathbf{F}\}$ (B is the candidate key!)

Step 4: Check Lossless Condition

- $R_1 - R_2 = \{D, E\}$ – Does B^+ contain $\{D, E\}$? **YES**
- $R_2 - R_1 = \{A, C, F\}$ – Does B^+ contain $\{A, C, F\}$? **YES**

Result: **LOSSLESS** (B determines everything!)

Lossless Check Summary Table

Decomposition	Intersection	Closure	Result
$R_1 = (A, B, C), R_2 = (A, D, E, F)$	$\{A\}$	$A^+ = \{A\}$	Not Lossless
$R_1 = (A, B, C, D), R_2 = (A, B, E, F)$	$\{A, B\}$	$(AB)^+ = \text{all}$	Lossless
$R_1 = (A, C, D), R_2 = (B, C, E, F)$	$\{C\}$	$C^+ = \{C\}$	Not Lossless
$R_1 = (B, D, E), R_2 = (A, B, C, F)$	$\{B\}$	$B^+ = \text{all}$	Lossless

Pattern: If intersection closure = all attributes, decomposition is lossless!

A.19.7.2 Dependency Preserving

Dependency Preserving (KEY CONCEPT)

A decomposition is **dependency preserving** if all FDs can be checked on individual relations (no joins needed).

Test: For each FD $X \to Y$, check if all attributes in X and Y exist together in at least one decomposed relation.

Dependency Preserving Example

$R = (A, B, C)$, $F = \{A \to B, B \to C\}$

Decomposition: $R_1 = (A, B)$, $R_2 = (B, C)$

- $A \to B$: can check in R_1 (has A, B)
- $B \to C$: can check in R_2 (has B, C)

Result: Dependency preserving!

NOT Dependency Preserving

$R = (A, B, C)$, $F = \{A \to B, B \to C, C \to A\}$

Decomposition: $R_1 = (A, B)$, $R_2 = (B, C)$

- $A \to B$: can check in R_1
- $B \to C$: can check in R_2
- $C \to A$: Cannot check in R_1 or R_2 alone!

Result: NOT dependency preserving!

Good Decomposition (REMEMBER)

A good decomposition is BOTH lossless AND dependency preserving.

A.19.7.3 3NF vs BCNF Trade-off

3NF vs BCNF Trade-off

Property	3NF	BCNF
Lossless Decomposition	Always achievable	Always achievable
Dependency Preserving	Always achievable	May lose some FDs

3NF vs BCNF (INTUITION)

3NF is sometimes "good enough" when BCNF would sacrifice dependency preservation.

A.19.8 Summary: Quick Reference

Understanding Dependency Types

Type	What It Means	Example	Problem
Full	Depends on entire key	$AB \to C$	None
Partial	Depends on PART of key	$A \to C$ (key=AB)	Redundancy
Transitive	Non-prime $\to$ Non-prime	$B \to C$ (B not key)	Chain redundancy

Normal Form Summary Table (CRITICAL)

NF	Rule	Allowed	Forbidden
1NF	Atomic values	-	Multi-valued attrs
2NF	No partial deps	Full key $\rightarrow$ non-prime	Part of key $\rightarrow$ non-prime
3NF	No transitive deps	Superkey $\rightarrow$ any, any $\rightarrow$ prime	Non-prime $\rightarrow$ non-prime
BCNF	LHS always superkey	Superkey $\rightarrow$ any	Non-superkey $\rightarrow$ anything

One-Line Rules (REMEMBER)

NF	One-Line Rule
1NF	No multi-values in cells
2NF	No partial dependencies (full key only on LHS)
3NF	LHS must be superkey OR RHS must be prime
BCNF	LHS must ALWAYS be superkey (no OR, no exceptions)

Normalization Quick Reference

Concept	Key Formula/Test
FD $X \rightarrow Y$	Same $X \Rightarrow$ same Y
Closure X^+	All attrs determined by X
Superkey K	$K^+ = R$
Candidate key	Minimal superkey
Prime attribute	Part of some candidate key
Non-prime attribute	Not part of any candidate key
Lossless	$(R_1 \cap R_2)^+ \supseteq R_1$ or R_2
Dep. preserving	All FDs checkable locally
3NF	X is superkey OR Y in candidate key
BCNF	X is superkey (no exceptions)

Exam Pattern (REMEMBER)

Exam pattern: "Given R and F, find candidate keys, check BCNF, test lossless decomposition."

A.20 Physical Design & Indexing (L08)

Overview

This section covers storage organization, B+ trees, and indexing strategies.

A.20.1 Storage Concepts

A.20.1.1 Storage Hierarchy

Storage Hierarchy

Level	Speed	Volatile?
Cache	Fastest	Yes
Main Memory (RAM)	Fast	Yes
Flash/SSD	Medium	No
Magnetic Disk (HDD)	Slow	No
Tape	Slowest	No

Block/Page (KEY CONCEPT)

Block (page) = basic unit of disk I/O. Goal: minimize block accesses.

Disk I/O Cost (INTUITION)

Disk I/O is SLOW. Reading one byte costs almost as much as reading a whole block.

A.20.1.2 File Organization Types

File Organization Types

Type	Good For	Bad For
Heap	Bulk insert	Search (must scan all)
Sequential	Range queries, sorted access	Insert (maintain order)
B+ tree	Balanced insert/search/range	–
Hash	Equality search	Range queries

A.20.2 Index Fundamentals

Index Concept (KEY CONCEPT)

An **index** enables fast lookups. Entry format: `| search-key | pointer |`.

A.20.2.1 Primary vs Secondary Index

Primary vs Secondary Index

Type	Definition	Properties
Primary (Clustering)	File sorted on index key	One per table, can be sparse
Secondary (Non-clustering)	File NOT sorted on key	Many per table, MUST be dense

A.20.2.2 Dense vs Sparse Index

Dense vs Sparse Index

Type	Definition	Trade-off
Dense	Entry for EVERY key value	Larger index, faster lookup
Sparse	Entry for SOME key values	Smaller index, requires sorted file

Secondary Index Rule (REMEMBER)

Secondary index MUST be dense (can't use sparse on unsorted data).

A.20.3 B+ Tree Structure

B+ Tree (CRITICAL)

B+ tree: balanced search tree with all data in leaves.

- All paths from root to leaf are of the **same length** (balanced)
- Leaf nodes are **linked together** for efficient range queries
- Internal nodes contain only keys (separators), not data

A.20.3.1 Node Structure

B+ Tree Node Structure

```
| P1 | K1 | P2 | K2 | ... | Kn-1 | Pn |

Ki = search keys (sorted: K1 < K2 < ... < Kn-1)
Pi = pointers (to children or records)
```

A.20.3.2 B+ Tree Properties (Degree n)

B+ Tree Properties (CRITICAL)

Know these formulas!

Internal Node:

- Children: between $\lceil n/2 \rceil$ and n
- Keys: between $\lceil n/2 \rceil - 1$ and $n - 1$
- Exception: root can have as few as 2 children

Leaf Node:

- Keys: between $\lceil (n-1)/2 \rceil$ and $n - 1$
- Linked together for range queries

A.20.3.3 Common Degree Values

Common Degree Values (CRITICAL)

n	Internal children	Internal keys	Leaf keys
4	[2, 4]	[1, 3]	[2, 3]
5	[3, 5]	[2, 4]	[2, 4]
6	[3, 6]	[2, 5]	[3, 5]

Quick formulas:
- Internal children: $[\lceil n/2 \rceil, n]$
- Internal keys: $[\lceil n/2 \rceil - 1, n - 1]$
- Leaf keys: $[\lceil (n-1)/2 \rceil, n - 1]$

Verifying n
- Internal children: $[\lceil 5/2 \rceil, 5] = [3, 5]$ ✓
- Internal keys: $[3 - 1, 5 - 1] = [2, 4]$ ✓
- Leaf keys: $[\lceil 4/2 \rceil, 4] = [2, 4]$ ✓

A.20.3.4 Navigation

B+ Tree Navigation

For key K at an internal node:
- Left pointer (before Ki): leads to keys $< K_i$
- Right pointer (after Ki): leads to keys $\geq K_i$

A.20.3.5 Search Operations

B+ Tree Search

Equality Search (find key K):
1. Start at root
2. At each internal node: follow pointer based on comparison with keys
3. Reach leaf node, scan for K

Range Search (find all keys in $[K_1, K_2]$):
1. Search for K_1 to find starting leaf
2. Follow leaf sibling pointers until K_2 is exceeded

Search Example (n

Tree: Root $[10, 20, 30]$ with leaves $[1^*, 5^*]$, $[12^*, 15^*, 17^*]$, $[21^*, 24^*]$, $[34^*, 35^*, 38^*]$
Search for 15*: $15 \geq 10$ and $15 < 20 \rightarrow$ follow 2nd pointer $\rightarrow$ leaf $[12^*, 15^*, 17^*] \rightarrow$ found!
Search for all $\geq$ 20*: Find 20 $\rightarrow$ leaf $[21^*, 24^*] \rightarrow$ follow sibling pointer $\rightarrow [34^*, 35^*, 38^*]$

A.20.4 B+ Tree Insert

B+ Tree Insert Algorithm (CRITICAL)

1. Find correct leaf
2. If leaf not full: insert in sorted order
3. If leaf full: **SPLIT**

A.20.4.1 Split Leaf

Split Leaf (CRITICAL)

When a leaf overflows (has n keys after insertion, where max is $n - 1$):
1. Create new leaf (right sibling)
2. First $\lfloor n/2 \rfloor$ keys stay in **left** leaf
3. Remaining $\lceil n/2 \rceil$ keys go to **right** leaf
4. **COPY UP** the **first key of the right leaf** to parent
5. If parent full, split parent

Which Key Gets Copied Up? (CRITICAL)

When splitting, the "middle" key is the **first key of the NEW (right) leaf**!
Example with n=4: Insert causes leaf to have [12, 14, 15, 16]

Left Leaf	Right Leaf	Key Copied Up
$[12, 14]$	$[15, 16]$	**15** (first of right)

Why 15 and not 14?
- Left gets $\lfloor 4/2 \rfloor = 2$ keys: positions 0,1 $\rightarrow$ [12, 14]
- Right gets $\lceil 4/2 \rceil = 2$ keys: positions 2,3 $\rightarrow$ [15, 16]
- The separator key must be $\geq$ all keys in left, and $\leq$ first key in right
- So we copy up **15** (the first key of the right leaf)

Leaf Split Examples

n=4, max leaf keys=3:

Keys After Insert	Left ($\lfloor n/2 \rfloor$)	Right ($\lceil n/2 \rceil$)	Copy Up
$[12, 15, 16, 17]$	$[12, 15]$	$[16, 17]$	16
$[12, 14, 15, 16]$	$[12, 14]$	$[15, 16]$	15
$[10, 11, 12, 13]$	$[10, 11]$	$[12, 13]$	12

n=5, max leaf keys=4:

Keys After Insert	Left ($\lfloor 5/2 \rfloor = 2$)	Right ($\lceil 5/2 \rceil = 3$)	Copy Up
$[2, 3, 5, 7, 8]$	$[2, 3]$	$[5, 7, 8]$	5
$[19, 20, 22, 23, 24]$	$[19, 20]$	$[22, 23, 24]$	22

A.20.4.2 Split Internal Node

Split Internal Node

1. Create new internal node
2. First $\lceil n/2 \rceil$ pointers stay in left node
3. **PUSH UP** middle key (don't keep copy—key is removed from node)
4. Remaining pointers go to right node
5. If root splits, create new root (tree grows taller)

Copy-up vs Push-up (REMEMBER)

Split Type	Action	Middle Key
Leaf split	**COPY UP**	Key stays in leaf AND goes to parent
Internal split	**PUSH UP**	Key moves to parent (removed from node)

Complete Insert Example: Insert 8* (n

Initial tree (n=5): Root $[13, 17, 24, 30]$ with 5 leaves
Step 1: Find correct leaf
- $8 < 13 \rightarrow$ follow first pointer
- Reach leaf $[2^*, 3^*, 5^*, 7^*]$ (already full—has 4 keys, max is $n - 1 = 4$)

Step 2: Insert causes overflow, split leaf

Keys after insert	Left leaf	Right leaf	Copy up
$[2, 3, 5, 7, 8]$	$[2^*, 3^*]$	$[5^*, 7^*, 8^*]$	**5**

Step 3: Parent overflows, split internal node
- Parent becomes $[5, 13, 17, 24, 30]$ (5 keys—overflow!)
- Split: Left $[5, 13]$, Right $[24, 30]$, **Push up 17**

Step 4: Create new root
- New root with single key [17]
- Left child $[5, 13]$, Right child $[24, 30]$
- Tree height increased by 1

A.20.5 B+ Tree Delete

B+ Tree Delete Algorithm (CRITICAL)

1. Find and delete from leaf
2. If underflow: try **redistribute**, else **merge**

A.20.5.1 Redistribute

Redistribute

- Borrow key from sibling (if sibling has extra)
- Update parent key (new separator)

A.20.5.2 Merge

Merge

- Combine with sibling
- Remove separator key from parent
- If parent underflows, recurse upward
- If root has only one child after merge, delete root

Delete Strategy (INTUITION)

Try redistribute first (cheaper). Merge only if redistribute impossible.

A.20.5.3 Underflow Thresholds

Underflow Thresholds (CRITICAL)

n	Node Type	Min Required	Underflow if
4	Leaf	$\lceil (4-1)/2 \rceil = 2$ keys	< 2 keys
4	Internal	$\lceil 4/2 \rceil = 2$ children	< 2 children
5	Leaf	$\lceil (5-1)/2 \rceil = 2$ keys	< 2 keys
5	Internal	$\lceil 5/2 \rceil = 3$ children	< 3 children
6	Leaf	$\lceil (6-1)/2 \rceil = 3$ keys	< 3 keys
6	Internal	$\lceil 6/2 \rceil = 3$ children	< 3 children

Key insight: For n=4 and n=5, leaf minimum is the same (2 keys), but internal minimum differs!

Delete with Redistribute: Delete 24* (n

Before: Root $[10, 20, 30]$, Leaf $[21^*, 24^*]$ has 2 keys
Step 1: Delete 24* $\rightarrow$ Leaf becomes $[21^*]$ (only 1 key—underflow!)
Step 2: Check sibling $[34^*, 35^*, 38^*]$ has 3 keys (can spare one)
Step 3: Redistribute—borrow 34* from sibling

Action	Left Leaf	Right Leaf
Before redistribute	$[21^*]$	$[34^*, 35^*, 38^*]$
After redistribute	$[21^*, 34^*]$	$[35^*, 38^*]$

Step 4: Update parent separator: $30 \rightarrow 35$

Delete with Merge: Delete 38* then Merge (n

Before: After redistribute, leaf $[35^*, 38^*]$
Step 1: Delete 38* $\rightarrow$ Leaf becomes $[35^*]$ (underflow!)
Step 2: Check sibling $[21^*, 34^*]$—only has 2 keys (minimum), can't redistribute
Step 3: Merge leaves: $[21^*, 34^*] + [35^*] \rightarrow [21^*, 34^*, 35^*]$
Step 4: Remove separator key (35) from parent

Cascade Delete: Delete 24* causing multiple merges (n

When a merge causes the parent to underflow, the merge **cascades upward**:

1. Delete from leaf causes leaf underflow
2. Merge leaf with sibling, remove separator from parent
3. Parent now underflows $\rightarrow$ merge parent with sibling

4. If root has only one child after merge, **delete root** (tree shrinks)
This can reduce tree height!

A.20.6 Hashing

Hash Function (KEY CONCEPT)

Hash function: $h(\text{search_key}) \rightarrow \text{bucket_address}$

A.20.6.1 Concepts

Hashing Concepts

- **Bucket**: Unit of storage (usually one disk block)
- **Collision**: Different keys hash to same bucket
- **Overflow chaining**: Link overflow buckets

A.20.6.2 Hashing vs B+ Tree

Hashing vs B+ Tree Comparison

Aspect	Hashing	B+ Tree
Equality search	O(1) average	O(log n)
Range queries	NOT supported	Supported
Complexity	Simple	More complex
Growth handling	Fixed buckets (static)	Graceful

A.20.7 Summary: Quick Reference

Physical Design Quick Reference

Concept	Key Point
Block	Basic I/O unit, minimize accesses
Primary index	File sorted on key, can be sparse
Secondary index	File not sorted, MUST be dense
B+ Tree Properties (degree n)	
Internal node children	$[\lceil n/2 \rceil, n]$ (root: $[2, n]$)
Internal node keys	$[\lceil n/2 \rceil - 1, n - 1]$
Leaf node keys	$[\lceil (n-1)/2 \rceil, n - 1]$
B+ Tree Operations	
Search	Follow pointers: left $< K$, right $\geq K$
Insert overflow	Split: **COPY** up (leaf), **PUSH** up (internal)
Delete underflow	**Redistribute** first, then **merge**
Leaf split (n keys)	Left: $\lfloor n/2 \rfloor$, Right: $\lceil n/2 \rceil$, copy first of right
Index Choice	
Hash index	O(1) equality only, no range queries
B+ tree index	O(log n) equality + range queries

Common Exam Mistakes (CRITICAL)

1. **Wrong split key**: Always copy up the **first key of the RIGHT leaf**, not the last key of left
2. **Confusing copy-up vs push-up**: Leaf = COPY (key stays), Internal = PUSH (key moves)
3. **Wrong underflow threshold**: Remember leaf uses $(n-1)/2$, internal uses $n/2$
4. **Forgetting root exception**: Root can have as few as 2 children (or be a single leaf)
5. **Missing cascade**: Merge can propagate up the tree, possibly reducing height

A.21 Query Processing (L09)

Overview

This section covers query execution algorithms, cost estimation, and optimization.

A.21.1 Query Processing Overview

A.21.1.1 Processing Steps

Query Processing Steps

1. **Parsing**: Check syntax, verify relations exist
2. **Translation**: Convert to relational algebra expression
3. **Optimization**: Find efficient execution plan
4. **Evaluation**: Execute the plan

Cost Measurement (KEY CONCEPT)

Cost is measured in **disk block transfers** and **seeks**. Memory (buffer) size B is critical.

A.21.2 External Merge Sort

External Merge Sort (CRITICAL)

External sort handles data larger than memory. Two phases: create runs, merge runs.

A.21.2.1 Algorithm

External Sort Algorithm

Phase 1 - Create Runs:

1. Read B pages into memory
2. Sort internally
3. Write sorted run to disk
4. Repeat until all data processed

Phase 2 - Merge Runs:

1. Use B-1 input buffers + 1 output buffer
2. Merge B-1 runs at a time
3. Repeat until one sorted run remains

A.21.2.2 Cost Formula

External Sort Cost Formula (CRITICAL)

Memorize this formula!

$$\text{Total Cost} = 2N \times (\lceil \log_{B-1} \lceil N/B \rceil \rceil + 1)$$

Where:

- N = number of pages in relation
- B = number of buffer pages available
- $\lceil N/B \rceil$ = number of initial runs
- $\lceil \log_{B-1}(\text{runs}) \rceil$ = number of merge passes

Sort Cost Intuition (INTUITION)

Each pass reads and writes all N pages once (cost = 2N). We do (merge passes + 1) passes total.

A.21.2.3 Example Calculation

External Sort Example

Given: $N = 1000$ pages, $B = 5$ buffer pages

1. Initial runs $= \lceil 1000/5 \rceil = 200$ runs
2. Merge passes $= \lceil \log_4(200) \rceil = \lceil 3.82 \rceil = 4$
3. Total passes $= 4 + 1 = 5$
4. **Total cost** $= 2 \times 1000 \times 5 = 10,000$ **block transfers**

A.21.3 Join Algorithms

Join Algorithms (CRITICAL)

Join is the most expensive operation. Know all four algorithms and their costs!

A.21.3.1 Notation

Join Notation

Symbol	Meaning
n_r, n_s	Number of tuples in r, s
b_r, b_s	Number of blocks in r, s
B	Number of buffer pages

A.21.3.2 Nested-Loop Join (Simple)

Nested-Loop Join

```
for each tuple t_r in r:          (outer)
    for each tuple t_s in s:      (inner)
        if match: output (t_r, t_s)
```

Cost:

$$\text{Cost} = n_r \times b_s + b_r$$

Nested-Loop Intuition (INTUITION)

For EACH outer tuple, scan ALL inner blocks. Very expensive!

Outer Relation Choice (REMEMBER)

Put smaller relation as OUTER to reduce cost.

A.21.3.3 Block Nested-Loop Join

Block Nested-Loop Join

```
for each block B_r in r:          (outer)
    for each block B_s in s:      (inner)
        for each tuple t_r in B_r:
            for each tuple t_s in B_s:
                if match: output
```

Cost:

$$\text{Cost} = b_r \times b_s + b_r$$

Block NL Intuition (INTUITION)

For each BLOCK of outer, scan all inner blocks. Much better than tuple-based!

Optimized Block NL with B-2 outer blocks

$$\text{Cost} = \lceil b_r/(B-2) \rceil \times b_s + b_r$$

Where B-2 blocks used for outer (1 for inner, 1 for output).

Block NL Example

Given: $b_r = 800$, $b_s = 1500$, $B = 52$ buffers

Using r as outer:

$$\text{Cost} = \lceil 800/50 \rceil \times 1500 + 800 = 16 \times 1500 + 800 = 24,800$$

Using s as outer:

$$\text{Cost} = \lceil 1500/50 \rceil \times 800 + 1500 = 30 \times 800 + 1500 = 25,500$$

Smaller Outer (REMEMBER)

Put SMALLER relation as outer!

A.21.3.4 Sort-Merge Join

Sort-Merge Join

Algorithm:
1. Sort r on join attribute
2. Sort s on join attribute
3. Merge sorted relations (like merge sort)

Cost:

$$\text{Cost} = \text{sort}(r) + \text{sort}(s) + b_r + b_s$$

Sort-Merge Intuition (INTUITION)

After sorting, each relation scanned only once in merge phase.

When to Use Sort-Merge

When to use:
- Relations already sorted (or have index)
- Result needs to be sorted anyway
- Large relations where hash join won't fit

A.21.3.5 Hash Join

Hash Join

Algorithm:
1. **Partition phase**: Hash both relations into buckets
2. **Build phase**: Build hash table from smaller partition
3. **Probe phase**: Probe with larger partition

Cost:

$$\text{Cost} = 3(b_r + b_s)$$

(if no recursive partitioning needed)

Requirement: At least one partition must fit in memory.

A.21.3.6 Join Algorithm Comparison

Join Algorithm Comparison

Algorithm	Best Case	When to Use
Nested-loop	Small outer, indexed inner	Small relations
Block NL	$\lceil b_r/(B-2) \rceil \times b_s + b_r$	No index, limited memory
Sort-merge	$\approx 3(b_r + b_s)$	Already sorted
Hash join	$3(b_r + b_s)$	Equality joins, enough memory

A.21.4 Selection Algorithms

A.21.4.1 Selection Methods

Selection Methods

Method	Cost	Condition
Linear scan	b_r	No index
Primary index (equality)	$h_i + 1$	Clustering index on key
Primary index (range)	$h_i + b$	Clustering, b = matching blocks
Secondary index (equality)	$h_i + 1$ or $h_i + n$	Non-clustering

Where h_i = height of index (B+ tree levels).

Secondary Index Cost (REMEMBER)

Secondary index on non-key may require many disk accesses (one per matching tuple).

A.21.5 Cost Estimation

Cost Estimation (KEY CONCEPT)

Query optimizer uses statistics to estimate operation costs.

A.21.5.1 Catalog Statistics

Catalog Statistics

Statistic	Meaning
n_r	Number of tuples in relation r
b_r	Number of blocks containing tuples of r
l_r	Size of a tuple of r in bytes
f_r	Blocking factor (tuples per block)
$V(A, r)$	Number of distinct values for attribute A in r

A.21.6 Size Estimation

Size Estimation (CRITICAL)

Estimate output size to choose best plan.

A.21.6.1 Selection Size

Selection Size Estimation

Equality: $\sigma_{A=v}(r)$

$$\text{Size} = n_r / V(A, r)$$

Range: $\sigma_{A \leq v}(r)$

$$\text{Size} = n_r \times \frac{v - \min(A, r)}{\max(A, r) - \min(A, r)}$$

Selection Estimation (INTUITION)

Assume uniform distribution of values.

A.21.6.2 Join Size

Join Size Estimation

Cartesian Product:

$$|r \times s| = n_r \times n_s$$

Natural Join ($\mathbf{R \cap S = \emptyset}$):

$$|r \bowtie s| = n_r \times n_s$$

Natural Join (common attribute A):

If A is key of R and FK in S:

$$|r \bowtie s| = n_s$$

General case:

$$|r \bowtie s| = \frac{n_r \times n_s}{\max(V(A, r), V(A, s))}$$

FK Join Size (INTUITION)

Each S tuple matches exactly one R tuple.

A.21.6.3 Projection Size

Projection Size

$$|\Pi_A(r)| = V(A, r)$$

(for projection without duplicates)

A.21.7 Query Optimization

Query Optimization Goal (KEY CONCEPT)

Goal: find equivalent expression with lowest cost.

A.21.7.1 Equivalence Rules

Equivalence Rules

1. **Selection cascade**: $\sigma_{\theta_1 \wedge \theta_2}(E) = \sigma_{\theta_1}(\sigma_{\theta_2}(E))$
2. **Selection commutative**: $\sigma_{\theta_1}(\sigma_{\theta_2}(E)) = \sigma_{\theta_2}(\sigma_{\theta_1}(E))$
3. **Projection cascade**: Only outermost needed (if subset)
4. **Join commutative**: $r \bowtie s = s \bowtie r$
5. **Join associative**: $(r \bowtie s) \bowtie t = r \bowtie (s \bowtie t)$

A.21.7.2 Heuristic Optimization

Heuristic Optimization (CRITICAL)

Apply these transformations early:

1. **Push selections down**: Reduce tuple count early
2. **Push projections down**: Reduce tuple width early
3. **Combine selections with Cartesian products**: Create joins
4. **Order joins**: Smallest intermediate results first

Most Important Optimization (REMEMBER)

Selection before join is the most important optimization!

A.21.8 Pipelining

Pipelining (KEY CONCEPT)

Pass results directly to next operation without writing to disk.

Materialization vs Pipelining

Materialization vs Pipelining:

Method	Approach	Trade-off
Materialization	Store intermediate results	More I/O, simpler
Pipelining	Stream tuples between operations	Less I/O, complex

Blocking Operators

Blocking operators (cannot pipeline):

- Sort (need all input before output)
- Hash join build phase
- Aggregation (GROUP BY)

A.21.9 Summary: Quick Reference

Query Processing Quick Reference

Operation	Key Cost Formula
External sort	$2N \times (\lceil \log_{B-1} \lceil N/B \rceil \rceil + 1)$
Simple nested-loop	$n_r \times b_s + b_r$
Block nested-loop	$\lceil b_r/(B-2) \rceil \times b_s + b_r$
Sort-merge join	sort(r) + sort(s) + $b_r + b_s$
Hash join	$3(b_r + b_s)$
Selection (equality)	$n_r/V(A, r)$ output tuples
Join size (FK)	n_s (if A is key of R, FK of S)

Exam Pattern (REMEMBER)

Exam pattern: "Given b_r, b_s, B, calculate join cost using different algorithms."

A.22 Transactions & Concurrency (L10)

Overview

This section covers transaction management, concurrency control, and locking protocols.

A.22.1 Transaction Fundamentals

Transaction (KEY CONCEPT)

A **transaction** is a unit of program execution that accesses and updates data items.

A.22.1.1 Example Transaction

Transaction Example

```
Transfer $50 from A to B:
1. read(A)
2. A := A - 50
3. write(A)
4. read(B)
5. B := B + 50
6. write(B)
```

A.22.1.2 Transaction Operations

Transaction Operations

Operation	Effect
BEGIN	Start transaction
COMMIT	End successfully, make changes permanent
ROLLBACK/ABORT	Undo all changes, end transaction
SAVEPOINT name	Mark point for partial rollback

A.22.2 ACID Properties

ACID Properties (CRITICAL)

Memorize ACID - always tested!

Property	Meaning	Ensures
Atomicity	All or nothing	Partial execution impossible
Consistency	Valid state to valid state	Integrity constraints hold
Isolation	Transactions don't interfere	Concurrent = serial
Durability	Committed = permanent	Survives crashes

ACID Intuition (INTUITION)

A = all or nothing. C = valid → valid. I = can't see intermediate states. D = committed = permanent.

A.22.2.1 Which Property is Violated?

Violation Patterns (REMEMBER)

Common exam pattern:

Scenario	Violated Property
Partial update visible	Atomicity
Constraint broken after commit	Consistency
Lost update, dirty read	Isolation
Data lost after commit	Durability

A.22.3 Concurrency Problems

Concurrency Problems (KEY CONCEPT)

Without proper control, concurrent transactions cause problems:

Problem	Description
Lost Update	Two updates to same item, one lost
Dirty Read	Read uncommitted data that gets rolled back
Non-repeatable Read	Same query returns different results
Phantom	New rows appear/disappear between queries

A.22.4 Schedules and Serializability

A.22.4.1 Definitions

Schedule Types (KEY CONCEPT)

A **schedule** specifies the order of operations from concurrent transactions.

Schedule Type	Definition
Serial	Transactions execute one after another (no interleaving)
Concurrent	Operations from different transactions interleaved
Serializable	Equivalent to some serial schedule

Serializability Goal (INTUITION)

Serial schedules are always correct. Goal: allow concurrency but behave as if serial.

A.22.5 Conflict Serializability

Conflict Definition (CRITICAL)

Two operations **conflict** if:
1. From different transactions
2. Access same data item
3. At least one is a WRITE

A.22.5.1 Conflict Table

Conflict Table

Op 1	Op 2	Conflict?
R(X)	R(X)	NO (both reads)
R(X)	W(X)	YES
W(X)	R(X)	YES
W(X)	W(X)	YES

Conflict Rule (REMEMBER)

Two reads never conflict. Any write creates conflict.

A.22.5.2 Definition

Conflict Serializable

A schedule is **conflict serializable** if it can be transformed to a serial schedule by swapping non-conflicting adjacent operations.

A.22.6 Precedence Graph

Precedence Graph (CRITICAL)

Primary method to test conflict serializability!

A.22.6.1 Construction Algorithm

Precedence Graph Construction

1. Create one node for each transaction
2. For each pair of conflicting operations:
   - If T_i's operation comes BEFORE T_j's operation
   - Add edge $T_i \to T_j$
3. Check for cycles

A.22.6.2 Result Interpretation

Precedence Graph Result

Graph Property	Conclusion
No cycle (acyclic)	Conflict serializable
Has cycle	NOT conflict serializable

If acyclic: topological sort gives equivalent serial schedule.

Precedence Graph Intuition (INTUITION)

Edge $T_i \rightarrow T_j$ means "T_i must come before T_j in any equivalent serial schedule." Cycle means contradictory requirements.

A.22.6.3 Example

Precedence Graph Example

```
Schedule:
T1: W(A);           W(C); commit.
T2:      W(B);           R(C);      commit.
T3:           R(B); commit.

Conflicts:
- T1.W(C) before T2.R(C) --> T1 -> T2
- T2.W(B) before T3.R(B) --> T2 -> T3

Precedence Graph: T1 -> T2 -> T3

No cycle --> CONFLICT SERIALIZABLE
Equivalent serial: T1, T2, T3
```

A.22.7 Lock-Based Concurrency Control

Locking (KEY CONCEPT)

Locks prevent conflicting operations from executing concurrently.

A.22.7.1 Lock Types

Lock Types

Lock	Purpose	Compatibility
Shared (S-lock)	Reading	Multiple S-locks OK
Exclusive (X-lock)	Writing	Only one X-lock

A.22.7.2 Lock Compatibility Matrix

Lock Compatibility Matrix (CRITICAL)

Memorize this matrix!

	S held	X held
Request S	Grant	Wait
Request X	Wait	Wait

Lock Compatibility (INTUITION)

Multiple readers OK. Any writer blocks everyone else.

A.22.7.3 Lock Operations

Lock Operations

```
lock-S(Q)    -- request shared lock on Q
lock-X(Q)    -- request exclusive lock on Q
unlock(Q)    -- release lock on Q
```

A.22.8 Two-Phase Locking (2PL)

Two-Phase Locking (CRITICAL)

2PL guarantees conflict serializability!

A.22.8.1 The Two Phases

2PL Phases

Phase	Allowed	Not Allowed
Growing	Acquire locks	Release locks
Shrinking	Release locks	Acquire locks

Lock Point (KEY CONCEPT)

Lock point: moment when transaction holds all its locks (transition point).

A.22.8.2 2PL Examples

Valid 2PL

Valid 2PL:

```
lock-X(A), lock-X(B), lock-X(C), unlock(A), unlock(B), unlock(C)
|-------- growing --------|-------- shrinking --------|
```

NOT 2PL (violation)

NOT 2PL (violation):

```
lock-X(A), lock-X(B), unlock(B), lock-X(C), unlock(A), unlock(C)
                      ^^^^^^^^   ^^^^^^^^^
                      released   acquired after release!
```

2PL Rule (REMEMBER)

Once you release ANY lock, you cannot acquire ANY new lock.

A.22.8.3 2PL Variants

2PL Variants

Variant	Rule
Basic 2PL	Growing then shrinking
Strict 2PL	Hold X-locks until commit/abort
Rigorous 2PL	Hold ALL locks until commit/abort

Strict 2PL (INTUITION)

Strict/rigorous 2PL prevents cascading rollbacks.

A.22.9 Deadlock

Deadlock (KEY CONCEPT)

Deadlock: circular wait where each transaction waits for a lock held by another.

A.22.9.1 Example

Deadlock Example

```
T1: holds X(A), waiting for X(B)
T2: holds X(B), waiting for X(A)
--> Neither can proceed = DEADLOCK
```

A.22.9.2 Wait-For Graph

Wait-For Graph (CRITICAL)

Method to detect deadlock.

Construction:

- Vertex for each transaction
- Edge $T_i \to T_j$ if T_i waits for lock held by T_j

Rule: Deadlock exists $\Leftrightarrow$ wait-for graph has a CYCLE.

A.22.9.3 Example Wait-For Graph

Wait-For Graph Example

```
T1 waits for B (held by T2)  --> T1 -> T2
T2 waits for A (held by T1)  --> T2 -> T1
T3 waits for A (held by T1)  --> T3 -> T1

Graph: T1 <-> T2, T3 -> T1
       ^^^^^^^^
       Cycle! DEADLOCK between T1 and T2
```

A.22.9.4 Deadlock Handling

Deadlock Handling

Approach	Method
Prevention	Request all locks upfront; or lock in fixed order
Detection	Periodically check wait-for graph for cycles
Timeout	Abort if waiting too long
Recovery	Roll back victim transaction

A.22.9.5 Victim Selection

Victim Selection

Consider:

- How long transaction has run
- How many locks held
- How much work to redo
- Avoid starvation (same victim repeatedly)

A.22.10 Deadlock Exercise Pattern

Deadlock Exercise Pattern (REMEMBER)

From L10 exercises - common exam pattern:

```
Given transactions with 2PL:
T1: Lock-S(A); R(A); Lock-X(B); R(B); W(B); Unlock(A); Unlock(B).
T2: Lock-S(B); R(B); Lock-X(A); R(A); W(A); Unlock(B); Unlock(A).

Possible interleaving:
T1: Lock-S(A);              Lock-X(B); -- BLOCKED (T2 has S(B))
T2:            Lock-S(B);              Lock-X(A); -- BLOCKED (T1 has S(A))

Wait-for graph:
T1 -> T2 (T1 waits for B held by T2)
T2 -> T1 (T2 waits for A held by T1)

Cycle: T1 <-> T2 --> DEADLOCK
```

A.22.11 Summary: Quick Reference

Transactions Quick Reference

Concept	Key Point
ACID	Atomicity, Consistency, Isolation, Durability
Conflict	Different transactions, same item, ≥ 1 write
Precedence graph	Acyclic = conflict serializable
S-lock	For reading, multiple compatible
X-lock	For writing, exclusive
2PL	Growing then shrinking phase
Deadlock	Cycle in wait-for graph

Exam Patterns (REMEMBER)

Exam patterns:
- "Draw precedence graph. Is schedule conflict serializable?"
- "Does this transaction sequence follow 2PL?"
- "Draw wait-for graph. Is there deadlock?"

A.23 Recovery (L11)

This section covers database recovery techniques to ensure atomicity and durability.

A.23.1 Recovery Purpose

Recovery Purpose (KEY CONCEPT)

Recovery ensures ATOMICITY and DURABILITY despite failures.
- Committed transactions must persist (durability)
- Uncommitted transactions must have no effect (atomicity)

A.23.2 Failure Types

Failure Types (CRITICAL)

Three types of failures require different recovery approaches.

Failure Type	Cause	Recovery
Transaction failure	Logic error, abort, deadlock	UNDO that transaction
System crash	Power failure, OS crash	REDO committed, UNDO uncommitted
Disk failure	Storage destroyed	Restore from backup

What Survives a Crash?

Storage	After Crash
Main memory (buffers)	LOST
Disk data	Preserved (but may be outdated)
Log file (on disk)	Preserved

Crash Survival (INTUITION)

Crash loses all memory. Log (on disk) survives and tells us what happened.

A.23.3 Write-Ahead Logging (WAL)

WAL Rules (CRITICAL)

WAL is the foundation of recovery. Two rules:

Rule 1 (Commit): Before transaction commits, ALL its log records must be written to stable storage.

Rule 2 (Data): Before modified page written to disk, ALL log records for that page must be written to stable storage.

WAL Intuition (INTUITION)

"Write the log BEFORE writing the data." If we crash after log but before data, we can redo from log. If we crash before log, we have nothing to recover anyway.

A.23.4 Log Record Structure

Update Record Format (CRITICAL)

```
[TID, DID, old, new]
  |    |    |    |
  |    |    |    +-- value AFTER update (for REDO)
  |    |    +------- value BEFORE update (for UNDO)
  |    +------------ data item identifier
  +----------------- transaction identifier
```

Control Records

```
[TID start]   -- transaction started
[TID commit]  -- transaction committed
```

```
[TID abort]   -- transaction aborted
```

Example Log

```
[T1 start]
[T1, B, 300, 400]    <- T1 changed B: 300 -> 400
[T1, A, 100, 200]    <- T1 changed A: 100 -> 200
[T1 commit]
[T2 start]
[T2, C, 50, 70]      <- T2 changed C: 50 -> 70
<-- CRASH (T2 not committed)
```

UNDO vs REDO Values (REMEMBER)

Old value for UNDO, new value for REDO.

A.23.5 Recovery Algorithm

Two-Phase Recovery (CRITICAL)

Two phases: REDO then UNDO.

Phase 1: REDO (Forward Scan)

Scan log from START to END:
1. `[TID start]`: Add TID to **undo list**
2. `[TID, DID, old, new]`: Apply **new** value to DID
3. `[TID commit]`: Remove TID from undo list
4. `[TID abort]`: Remove TID from undo list

Result: Database reflects ALL operations. Undo list contains uncommitted transactions.

Phase 2: UNDO (Backward Scan)

Scan log from END to START (only for transactions in undo list):
1. `[TID, DID, old, new]`: Restore **old** value, write compensation record
2. `[TID start]`: Write `[TID abort]`, remove from undo list

Stop when: Undo list is empty.

Compensation Log Record

```
[TID, DID, old]
```

Records that we undid an operation. Ensures recovery is idempotent (can crash during recovery and restart).

A.23.6 Complete Recovery Example

Recovery Example Pattern (CRITICAL)

Memorize this pattern for exams!

Initial State and Log

```
Initial: A=100, B=200, C=50, D=60, E=120

Log at crash:
[T1 start]
[T1, B, 200, 400]
[T1, C, 50, 100]
[T2 start]
[T2, E, 120, 480]
[T1, A, 100, 320]
[T1 commit]
[T2, A, 320, 520]
[T2, D, 60, 530]
<-- CRASH (T2 not committed)
```

Phase 1: REDO (Forward)

Log Record	Action	Undo List
[T1 start]	–	{T1}
[T1, B, 200, 400]	B = 400	{T1}
[T1, C, 50, 100]	C = 100	{T1}
[T2 start]	–	{T1, T2}
[T2, E, 120, 480]	E = 480	{T1, T2}
[T1, A, 100, 320]	A = 320	{T1, T2}
[T1 commit]	–	{T2}
[T2, A, 320, 520]	A = 520	{T2}
[T2, D, 60, 530]	D = 530	{T2}

After REDO: A=520, B=400, C=100, D=530, E=480, Undo list = {T2}

Phase 2: UNDO - Backward Scan for T2

Log Record	Action	Compensation
[T2, D, 60, 530]	D = 60	[T2, D, 60]
[T2, A, 320, 520]	A = 320	[T2, A, 320]
[T2, E, 120, 480]	E = 120	[T2, E, 120]
[T2 start]	–	[T2 abort]

Final Result

```
Final database: A=320, B=400, C=100, D=60, E=120

T1 effects preserved: B=400, C=100, A=320 (committed)
T2 effects removed: A, D, E restored to pre-T2 values

New log records:
[T2, D, 60]      (compensation)
[T2, A, 320]     (compensation)
[T2, E, 120]     (compensation)
[T2 abort]
```

A.23.7 Checkpointing

Checkpointing (KEY CONCEPT)

Checkpoints reduce recovery time by limiting how far back to scan.

Checkpoint process:

1. Write all dirty buffers to disk
2. Write checkpoint record to log
3. Record active transactions

Recovery with checkpoint:

- Start REDO from checkpoint (not beginning of log)
- Only consider transactions active at or after checkpoint

A.23.8 Summary: Quick Reference

Quick Reference Table (REMEMBER)

Concept	Key Point
WAL	Log before data
Log record	[TID, DID, old, new]
REDO	Forward scan, apply new values
UNDO	Backward scan, restore old values
Undo list	Transactions started but not committed/aborted
Compensation	[TID, DID, old] - records the undo

Exam Pattern (CRITICAL)

1. Given log at crash, show database state after recovery
2. Forward pass: apply new values, build undo list
3. Backward pass: restore old values for undo list transactions
4. Show compensation records and final abort records

PART B: Reference Tables & Quick Summaries

B.1 Relational Algebra Reference

B.1.1 Operators at a Glance

Relational Algebra Operators (CRITICAL)

Operator	Symbol	Purpose
Selection	$\sigma_\theta(R)$	Filter rows by condition
Projection	$\Pi_{A_1,\ldots,A_n}(R)$	Select columns
Union	$R \cup S$	Combine tuples (duplicates removed)
Difference	$R - S$	Tuples in R but not in S
Cartesian Product	$R \times S$	All combinations
Natural Join	$R \bowtie S$	Join on common attributes
Theta Join	$R \bowtie_\theta S$	Join with condition
Rename	$\rho_S(R)$ or $\rho_{A/B}(R)$	Rename relation or attribute

B.1.2 Set vs Bag Semantics

Set vs Bag Semantics (REMEMBER)

Context	Duplicates	Notes
Relational Algebra	Removed (set)	Mathematical definition
SQL (default)	Kept (bag)	Use DISTINCT to remove

B.1.3 Aggregation

Aggregation (KEY CONCEPT)

$$_{G_1,\ldots,G_n}\mathcal{G}_{F_1(A_1),\ldots,F_m(A_m)}(R)$$

Aggregate Functions: SUM, AVG, COUNT, MIN, MAX

B.1.4 Division

Division (CRITICAL)

$$R \div S = \Pi_{R-S}(R) - \Pi_{R-S}((\Pi_{R-S}(R) \times S) - R)$$

Use for "all" queries (e.g., students taking ALL courses).

B.1.5 Key Properties

Key Properties (REMEMBER)

- σ and $\times$ are commutative
- $\bowtie$ is commutative and associative
- Selection can be pushed down: $\sigma_\theta(R \bowtie S) = \sigma_\theta(R) \bowtie S$ (if θ only involves R)

B.2 SQL Reference

B.2.1 Basic Query Structure

Basic Query Structure (CRITICAL)

```
SELECT [DISTINCT] columns
FROM tables
[WHERE condition]
[GROUP BY columns]
[HAVING condition]
[ORDER BY columns [ASC|DESC]];
```

B.2.2 Set Operations

Set Operations

Operation	Duplicates
UNION	Removed
UNION ALL	Kept
INTERSECT	Removed
EXCEPT	Removed

B.2.3 NULL Handling

NULL Handling (CRITICAL)

Expression	Result
NULL = NULL	UNKNOWN
NULL AND TRUE	UNKNOWN
NULL OR TRUE	TRUE
NULL OR FALSE	UNKNOWN

(REMEMBER) Test with: `IS NULL`, `IS NOT NULL`

B.2.4 Join Types

Join Types (KEY CONCEPT)

Join	Result
INNER JOIN	Matching rows only
LEFT OUTER JOIN	All left + matching right (NULL if no match)
RIGHT OUTER JOIN	All right + matching left
FULL OUTER JOIN	All from both sides
NATURAL JOIN	Join on same-named columns

B.2.5 Aggregate Functions

Aggregate Functions (REMEMBER)

`COUNT(*)`, `COUNT(col)`, `SUM(col)`, `AVG(col)`, `MIN(col)`, `MAX(col)`

Note: Aggregates ignore NULLs (except COUNT(*))

B.2.6 Subquery Operators

Subquery Operators (CRITICAL)

Operator	Meaning
IN	Value in subquery result
NOT IN	Value not in subquery result
EXISTS	Subquery returns ≥ 1 row
NOT EXISTS	Subquery returns 0 rows
ALL	Compare to all values
SOME/ANY	Compare to at least one

B.2.7 Constraints

Constraints (KEY CONCEPT)

```
PRIMARY KEY (cols)
FOREIGN KEY (col) REFERENCES table(col)
    ON DELETE CASCADE | SET NULL | SET DEFAULT
    ON UPDATE CASCADE | SET NULL | SET DEFAULT
UNIQUE (cols)
NOT NULL
CHECK (condition)
```

B.3 Normalization Reference

B.3.1 Armstrong's Axioms

Armstrong's Axioms (CRITICAL)

Axiom	Rule
Reflexivity	If $\beta \subseteq \alpha$, then $\alpha \to \beta$
Augmentation	If $\alpha \to \beta$, then $\alpha\gamma \to \beta\gamma$
Transitivity	If $\alpha \to \beta$ and $\beta \to \gamma$, then $\alpha \to \gamma$
Union	If $\alpha \to \beta$ and $\alpha \to \gamma$, then $\alpha \to \beta\gamma$
Decomposition	If $\alpha \to \beta\gamma$, then $\alpha \to \beta$ and $\alpha \to \gamma$

B.3.2 Closure Algorithm

Closure Algorithm (KEY CONCEPT)

```
result := alpha
while (result changes) do
    for each FD beta -> gamma in F do
        if beta is subset of result then
            result := result UNION gamma
return result   (this is alpha+)
```

B.3.3 Key Definitions

Key Definitions (CRITICAL)

Term	Definition
Superkey K	$K^+ = R$ (determines all attributes)
Candidate Key	Minimal superkey
Prime Attribute	Part of some candidate key

(REMEMBER) Trick: Attributes never on RHS MUST be in every candidate key.

B.3.4 Normal Forms

Normal Form Requirements (CRITICAL)

Form	Requirement for each FD $\alpha \to \beta$
BCNF	α is superkey
3NF	α is superkey OR β in candidate key

B.3.5 Decomposition Tests

Decomposition Tests (CRITICAL)

Lossless: $(R_1 \cap R_2)^+ \supseteq R_1$ OR $(R_1 \cap R_2)^+ \supseteq R_2$
Dependency Preserving: All FDs can be checked locally (no joins needed)

B.3.6 Lossless Decomposition Test

Lossless Test Formula (CRITICAL)

$\{R_1, R_2\}$ is **lossless** if:

$$(R_1 \cap R_2)^+ \supseteq R_1 \quad \text{OR} \quad (R_1 \cap R_2)^+ \supseteq R_2$$

In words: Common attributes must be a superkey of at least one decomposed relation.

Lossless Test Example (Exam Pattern)

Given: $R_1 = (A, B, C, D)$, $R_2 = (A, B, E, F)$, FDs: $AB \to C$, $B \to A$, $ABC \to D$, $D \to EF$
Step 1: Find $R_1 \cap R_2 = \{A, B\}$
Step 2: Compute $(AB)^+$:

- Start: $\{A, B\}$
- $B \to A$: already have A
- $AB \to C$: add C $\to \{A, B, C\}$
- $ABC \to D$: add D $\to \{A, B, C, D\}$
- $D \to EF$: add E,F $\to \{A, B, C, D, E, F\}$

Step 3: Check: $(AB)^+ = \{A, B, C, D, E, F\} \supseteq R_1 = \{A, B, C, D\}$ ✓
Result: Lossless!

B.3.7 BCNF vs 3NF

BCNF vs 3NF Trade-off (REMEMBER)

	Lossless	Dep. Preserving
BCNF	Always	May fail
3NF	Always	Always

B.4 B+ Tree Reference

B.4.1 Node Capacity (Degree n)

Node Capacity Rules (CRITICAL)

Node Type	Min	Max
Internal children	$\lceil n/2 \rceil$	n
Internal keys	$\lceil n/2 \rceil - 1$	$n - 1$
Leaf keys	$\lceil (n-1)/2 \rceil$	$n - 1$

(REMEMBER) Exception: Root can have as few as 2 children.

B.4.2 Common Degree Values

Common Degree Values Quick Reference

n	Internal min/max	Internal keys	Leaf keys
4	2–4 children	1–3	2–3
5	3–5 children	2–4	2–4
6	3–6 children	2–5	3–5

B.4.3 Insert Algorithm

Insert Algorithm (KEY CONCEPT)

1. Find correct leaf
2. If not full: insert in sorted order
3. If full: SPLIT

Split Rules (CRITICAL)

Split Leaf: First $\lceil n/2 \rceil$ stay, rest go to new. **COPY UP** middle key.
Split Internal: First $\lceil n/2 \rceil$ pointers stay. **PUSH UP** middle key (move, don't copy).

B.4.4 Delete Algorithm

Delete Algorithm (KEY CONCEPT)

1. Find and delete from leaf
2. If underflow: try REDISTRIBUTE, else MERGE

Delete Operations (REMEMBER)

Redistribute: Borrow from sibling, update parent separator.
Merge: Combine with sibling, remove parent separator.

B.4.5 Underflow Conditions

Underflow Check (CRITICAL)

Leaf underflow: keys $< \lceil (n-1)/2 \rceil$
Internal underflow: pointers $< \lceil n/2 \rceil$
Example (n=4):

- Leaf underflow: keys $< \lceil 3/2 \rceil = 2$
- Internal underflow: pointers $< \lceil 4/2 \rceil = 2$

On underflow: Try redistribute from sibling, else merge.

B.4.6 Index Types

Index Types (CRITICAL)

Type	File Order	Sparse OK?
Primary (Clustering)	Sorted on key	Yes
Secondary	Not sorted	No (must be dense)

B.5 Query Cost Reference

B.5.1 Notation

Notation (KEY CONCEPT)

n_r, n_s	Number of tuples
b_r, b_s	Number of blocks
B	Buffer pages available
$V(A, r)$	Distinct values of A in r

B.5.2 External Sort

External Sort Cost (CRITICAL)

$$\text{Cost} = 2N \times (\lceil \log_{B-1} \lceil N/B \rceil \rceil + 1)$$

Where N = pages, B = buffers.

Phase 1 and Phase 2 Breakdown (KEY CONCEPT)

Phase 1 (Create sorted runs):
- Initial runs: $\lceil N/B \rceil$
- I/O cost: $2N$ (read all + write all)

Phase 2 (Merge passes):
- Number of passes: $\lceil \log_{B-1}(\text{initial runs}) \rceil$
- I/O per pass: $2N$ (read all + write all)

Example: 50,000 pages, 300 buffers
- Phase 1 runs: $\lceil 50000/300 \rceil = 167$
- Phase 1 I/O: $2 \times 50000 = 100,000$
- Phase 2 passes: $\lceil \log_{299}(167) \rceil = 1$

B.5.3 Join Algorithms

Join Algorithm Costs (CRITICAL)

Algorithm	Cost (block transfers)
Nested-loop	$n_r \times b_s + b_r$
Block nested-loop	$b_r \times b_s + b_r$
Block NL (optimized)	$\lceil b_r/(B-2) \rceil \times b_s + b_r$
Sort-merge	sort(r) + sort(s) + $b_r + b_s$
Hash join	$3(b_r + b_s)$

Join Optimization (REMEMBER)

Rule: Put SMALLER relation as outer in nested-loop joins.

B.5.4 Selection Size Estimation

Selection Size Estimation

Query Type	Estimated Size
$\sigma_{A=v}(r)$	$n_r/V(A, r)$
$\sigma_{A \leq v}(r)$	$n_r \times \frac{v - \min(A)}{\max(A) - \min(A)}$

B.5.5 Join Size Estimation

Join Size Estimation (KEY CONCEPT)

Case	Estimated Size
Cartesian ($R \times S$)	$n_r \times n_s$
A is key of R, FK in S	n_s
General case	$\frac{n_r \times n_s}{\max(V(A,r),V(A,s))}$

B.5.6 Optimization Heuristics

Optimization Heuristics (REMEMBER)

1. Push selections down (reduce tuples early)
2. Push projections down (reduce width early)
3. Order joins by smallest intermediate results

B.6 ACID & Locking Reference

B.6.1 ACID Properties

Atomicity	All or nothing
Consistency	Valid state $\rightarrow$ valid state
Isolation	Concurrent = serial
Durability	Committed = permanent

B.6.2 Conflict Definition

Two operations conflict if: (1) different transactions, (2) same item, (3) at least one write.

	R	W
R	No	Yes
W	Yes	Yes

B.6.3 Precedence Graph

- Node per transaction
- Edge $T_i \rightarrow T_j$ if T_i operation precedes conflicting T_j operation
- **Acyclic = conflict serializable**

B.6.4 Lock Compatibility

	S held	X held
Request S	Grant	Wait
Request X	Wait	Wait

B.6.5 Two-Phase Locking (2PL)

Growing phase	Acquire locks only
Shrinking phase	Release locks only

Rule: Once you release any lock, cannot acquire new locks.

B.6.6 Wait-For Graph

- Edge $T_i \rightarrow T_j$ if T_i waits for lock held by T_j
- **Cycle = deadlock**

B.6.7 Recovery Log Format

```
[TID start]                -- begin
[TID, DID, old, new]       -- update
[TID commit] / [TID abort]
```

B.6.8 Recovery Algorithm

Phase	Direction	Action
REDO	Forward	Apply new values, build undo list
UNDO	Backward	Restore old values for undo list

B.6.9 Exam Traps

Recovery Order Trap (CRITICAL)

TRAP: "During log-based recovery we first do undo and then redo"
Answer: FALSE! Correct order is **REDO first, then UNDO**.

2PL Rule Trap (CRITICAL)

TRAP: "In 2PL, a transaction cannot obtain a lock if it has released a lock"
Answer: TRUE! Once in shrinking phase (any lock released), no new locks allowed.